

Criptografia — Guia de Estudo da Prova 1

PS-KatzL, PS3-1, PS4-2, PS5-2 e PS6-2

Jeroen van de Graaf — organização e notas de Arthur

Sumário

1	Escopo e fontes	3
1.1	Mapa de dependências	4
1.2	Planejamento por blocos	4
2	Parte I — Fundamentos formais de segurança	4
2.1	Esquemas de encriptação e corretude	4
2.2	Sigilo perfeito	5
2.2.1	One-Time Pad como exemplo completo	5
2.2.2	Lema 2.5 versus Definição 2.6	6
2.3	Segurança concreta e assintótica	6
2.3.1	Abordagem concreta	6
2.3.2	Abordagem assintótica	6
2.3.3	Soma de funções negligenciáveis	6
2.4	Definição 3.8 versus Definição 2.6	7
2.5	Segurança semântica	7
2.5.1	O experimento semântico, sem esconder os quantificadores	7
2.5.2	Da indistinguibilidade para a proteção de predicados	8
2.6	Geradores pseudoaleatórios	8
2.6.1	As distribuições são realmente diferentes	8
2.7	Pseudo-One-Time Pad	8
2.8	Provas por redução	8
2.8.1	Redução do pseudo-OTP, com a vantagem calculada	9
2.8.2	Híbridos	9
2.9	EAV e CPA	9
2.9.1	EAV: uma observação passiva	9
2.9.2	CPA: texto claro escolhido	9
2.9.3	Ataque explícito a uma cifra determinística	10
2.9.4	PRF e PRP: a abstração por trás dos modos	10
3	Parte II — Criptografia simétrica	10
3.1	Cifra de fluxo versus cifra de bloco	10
3.2	Rede de Feistel	11
3.2.1	Exemplo completo com quatro rodadas	11
3.3	DES e TripleDES	11
3.3.1	Por que 3DES não entrega simplesmente 168 bits	12
3.4	AES	12
3.4.1	Por que AES pode ser decriptado?	12
3.4.2	Anatomia de uma rodada do AES	13
3.4.3	Key schedule e tamanhos que não devem ser confundidos	13
3.5	Corpos finitos binários	13
3.5.1	Exemplo em $GF(2^4)$	14
3.5.2	Multiplicação em GCM	14
3.6	Modos de operação	15
3.6.1	ECB	15
3.6.2	CBC	15
3.6.3	Chained CBC	15
3.6.4	CTR	16
3.6.5	GCM	16
3.6.6	Verificação e estrutura de GHASH	16
3.6.7	Propagação de erro e maleabilidade	17
3.7	Resumo comparativo dos modos	17
4	Parte III — Aritmética modular, grupos e primos	17

4.1	Divisibilidade, máximo divisor comum e Bézout	17
4.1.1	Exemplo: inverso de 17 módulo 101	18
4.2	Aritmética modular	18
4.3	Square-and-Multiply	18
4.4	Grupos	19
4.4.1	Grupos aditivos e multiplicativos	19
4.4.2	Ordem de grupo e ordem de elemento	19
4.4.3	Grupo cíclico e gerador	19
4.5	Função de Euler	20
4.6	Teoremas de Fermat e Euler	20
4.6.1	Uso para testar composição	20
4.7	Teorema Chinês do Resto	20
4.7.1	Reconstrução explícita por CRT	21
4.8	Números primos e Teorema dos Números Primos	21
4.8.1	Quanto primos têm exatamente 100 algarismos?	21
4.9	Miller–Rabin	22
4.9.1	Duas rodadas executadas à mão	22
4.9.2	Dois significados de “probabilidade de erro”	23
4.9.3	Geração de um primo grande	23
4.10	Ordens: exemplos da lista	23
4.10.1	Grupo aditivo $\mathbb{Z}_{36}^+$	23
4.10.2	Grupo multiplicativo $\mathbb{Z}_{36}^*$	23
4.10.3	Grupo $\mathbb{Z}_{37}^*$	24
4.10.4	Tabela de potências módulo 17	24
5	Parte IV — Criptografia de chave pública	24
5.1	Visão geral	24
5.2	RSA	25
5.2.1	Geração de chaves	25
5.2.2	Exemplo pequeno de geração, encriptação e decriptação	25
5.2.3	Encriptação e decriptação textbook	25
5.2.4	Corretude quando $\gcd(m, n) = 1$	25
5.2.5	Corretude para todo $m \in \mathbb{Z}_n$	26
5.2.6	Decriptação acelerada por CRT	26
5.2.7	Por que $e = 65537$?	26
5.2.8	Fatorar versus inverter RSA	26
5.2.9	Conhecer $\varphi(n)$ implica fatorar $n = pq$	26
5.2.10	Mensagem não coprima com n	27
5.2.11	Procedimento para o exercício com primos de 12 algarismos	27
5.3	Por que textbook RSA não é seguro?	27
5.3.1	Determinismo	27
5.3.2	Maleabilidade multiplicativa	27
5.3.3	Mensagens pequenas e estrutura	27
5.4	OAEP	27
5.5	Diffie–Hellman	28
5.5.1	Exemplo completo e verificação	28
5.5.2	Por que “estabelecimento”, não “troca”?	28
5.5.3	Limitação: falta de autenticação	29
5.6	Problemas DLP, CDH e DDH	29
5.7	ElGamal	29
5.7.1	Geração	29
5.7.2	Encriptação	29
5.7.3	Decriptação	29
5.7.4	Exemplo completo	30
5.8	Escolha do grupo e safe primes	30
5.8.1	b ou c : qual usar?	30
5.9	Algoritmos para fatoração	30
5.9.1	Pollard–rho para fatoração	30
5.9.2	Exemplo executado	31
5.10	Algoritmos para log discreto	31
5.10.1	Busca exaustiva	31
5.10.2	Baby-Step/Giant-Step	31
5.10.3	Pollard–rho para DLP	31
5.10.4	Index Calculus	32
5.11	Curvas elípticas	32
5.11.1	Soma de pontos	32
5.11.2	Exemplo executado	33

5.11.3	ECDLP	33
5.12	Fatores de trabalho e tamanhos de chave	33
6	Parte V — Hashes, MACs e assinaturas	34
6.1	Objetivos diferentes	34
6.2	Função hash	34
6.2.1	Preimage resistance / one-wayness	34
6.2.2	Second-preimage resistance / weak collision resistance	34
6.2.3	Collision resistance / strong collision resistance	34
6.3	Paradoxo do aniversário	35
6.3.1	Exemplo com 36500 possibilidades	35
6.3.2	Exemplo das carteiras	35
6.4	Ataque de colisão contra hash	35
6.5	Hashes conhecidos	36
6.6	MAC	36
6.6.1	Construções perigosas	36
6.6.2	HMAC	36
6.6.3	CBC-MAC	36
6.7	Assinaturas digitais	36
6.7.1	MAC versus assinatura	37
6.8	Tipos de ataque contra assinaturas	37
6.8.1	Pelo acesso do atacante	37
6.8.2	Pelo objetivo alcançado	37
6.9	Assinatura RSA textbook	37
6.9.1	Exemplo com números pequenos	37
6.9.2	Fraquezas	37
6.10	Assinatura ElGamal	38
6.10.1	Exemplo com números pequenos	38
6.11	DSA	39
6.11.1	Assinatura	39
6.11.2	Verificação	39
6.11.3	Exemplo com números pequenos	39
6.11.4	Por que a otimização é segura?	40
6.12	Reutilização do nonce em DSA	40
6.13	ECDSA e padrões de curvas	40
7	Parte VI — Revisão orientada à prova	40
7.1	O que precisa de definição, mecanismo e ataque	40
7.2	Confusões que precisam estar resolvidas	41
7.3	Fórmulas que devem sair sem consulta	41
7.3.1	CBC	41
7.3.2	CTR	41
7.3.3	Euler	41
7.3.4	RSA	41
7.3.5	Diffie–Hellman	41
7.3.6	ElGamal	41
7.3.7	Aniversário	41
7.3.8	Assinatura ElGamal	41
7.3.9	DSA	42
7.4	Perguntas de autoavaliação	42
7.4.1	Fundamentos	42
7.4.2	Simétrica	42
7.4.3	Matemática	42
7.4.4	Chave pública	42
7.4.5	Hashes e autenticação	42
7.5	Referências por assunto	42

1 Escopo e fontes

Este guia cobre os assuntos pedidos em:

- **PS-KatzL:** definições de segurança, funções negligenciáveis, segurança semântica, geradores pseudoaleatórios, reduções, CPA e modos de operação;
- **PS3-1:** cifras de fluxo e de bloco, Feistel, DES/3DES, AES, corpos finitos, ECB, CBC, CTR e GCM;
- **PS4-2:** Euclides, inversos, primos, Fermat, Miller–Rabin, grupos, ordens, grupos cíclicos, CRT e função de Euler;
- **PS5-2:** RSA/OAEP, Diffie–Hellman, ElGamal, log discreto, Pollard–rho, Index Calculus, curvas elípticas e tamanhos de chave;

- **PS6-2:** hashes, paradoxo do aniversário, MACs e assinaturas RSA, ElGamal e DSA.

As fontes principais são *Introduction to Modern Cryptography*, 3ª edição, de Katz e Lindell, as listas da disciplina e os arquivos AULA_1/aula1.md e AULA_2/aula2.md. Para AES e parâmetros atuais, foram consultados também FIPS 197 e NIST SP 800-57.

1.1 Mapa de dependências

probabilidade	→	jogos de segurança	→	EAV/CPA, PRG/PRF/PRP
aritmética modular	→	grupos e primos	→	RSA, DH, ElGamal, ECC
PRP + $GF(2^m)$	→	AES e modos	→	GCM e autenticação
probabilidade + grupos	→	aniversário/Pollard	→	hashes e assinaturas

Não estude RSA antes de dominar inversos, $\varphi(n)$ e exponenciação modular. Não estude DSA antes de dominar grupos cíclicos, log discreto e ElGamal. Não decore os modos de operação antes de distinguir PRF, PRP, nonce e IV.

1.2 Planejamento por blocos

Bloco	Pré-requisitos	Conteúdo	Resultado que deve sair sem consulta
1	probabilidade básica	sigilo perfeito, vantagem, negligibilidade	comparar Def. 2.6 e Def. 3.8
2	bloco 1	PRG, redução, EAV e CPA	desenhar os experimentos
3	bloco 2	stream/block cipher, Feistel, DES, AES	explicar inversão e tamanhos
4	bloco 3	$GF(2^m)$, ECB, CBC, CTR, GCM	escrever todas as fórmulas
5	álgebra básica	Euclides, inversos, grupos, ordens, φ	fazer cálculos à mão
6	bloco 5	Fermat, Miller–Rabin, geração de primos	executar uma rodada
7	blocos 5–6	RSA e OAEP	gerar chave e provar corretude
8	bloco 5	DH, ElGamal, DLP e ECC	reproduzir os protocolos
9	probabilidade + bloco 8	aniversário, Pollard e Index Calculus	comparar fatores de trabalho
10	blocos 2–4 e 9	hash, MAC e assinaturas	distinguir objetivos e ataques

2 Parte I — Fundamentos formais de segurança

2.1 Esquemas de encriptação e corretude

Um esquema de encriptação de chave privada é uma tripla de algoritmos

$$\Pi = (\text{Gen}, \text{Enc}, \text{Dec}).$$

Com parâmetro de segurança λ :

$$k \leftarrow \text{Gen}(1^\lambda), \quad c \leftarrow \text{Enc}_k(m), \quad m' := \text{Dec}_k(c).$$

- Gen é probabilístico;
- Enc pode ser probabilístico ou usar nonce/IV;
- Dec normalmente é determinístico;
- **corretude** exige, para toda mensagem válida,

$$\Pr[\text{Dec}_k(\text{Enc}_k(m)) = m] = 1,$$

onde a probabilidade considera toda aleatoriedade de Gen e Enc.

Segurança e corretude são propriedades diferentes. Corretude diz que o destinatário recupera a mensagem; segurança diz o que um adversário não consegue aprender ou produzir.

2.2 Sigilo perfeito

Considere variáveis aleatórias M , K e $C = \text{Enc}_K(M)$. Antes de observar C , a probabilidade $\Pr[M = m]$ representa o conhecimento *a priori* sobre a mensagem. Depois de observar um cifrado c , esse conhecimento passa a ser $\Pr[M = m \mid C = c]$. Há sigilo perfeito quando, para toda distribuição de mensagens, toda mensagem com probabilidade positiva e todo cifrado que possa ocorrer,

$$\Pr[M = m \mid C = c] = \Pr[M = m].$$

Portanto, nem um adversário com poder computacional ilimitado atualiza sua crença sobre M ao ver C . Isso é uma afirmação de independência estatística, não de dificuldade de cálculo.

As duas caracterizações centrais são:

$$\Pr[M = m \mid C = c] = \Pr[M = m]$$

e, para todos $m, m' \in \mathcal{M}$ e $c \in \mathcal{C}$,

$$\Pr[\text{Enc}_K(m) = c] = \Pr[\text{Enc}_K(m') = c].$$

O Lema 2.5 prova que essas caracterizações são equivalentes. Para entender a direção menos óbvia, suponha que a distribuição do cifrado seja idêntica para qualquer mensagem. Fixado c , existe um valor p_c tal que

$$\Pr[C = c \mid M = m] = p_c$$

para todo m . Pela lei da probabilidade total,

$$\Pr[C = c] = \sum_{m'} \Pr[C = c \mid M = m'] \Pr[M = m'] = p_c.$$

Aplicando Bayes,

$$\Pr[M = m \mid C = c] = \frac{\Pr[C = c \mid M = m] \Pr[M = m]}{\Pr[C = c]} = \frac{p_c \Pr[M = m]}{p_c} = \Pr[M = m].$$

No sentido contrário, escolha a distribuição uniforme sobre $\mathcal{M}$. Se observar c não altera a probabilidade de nenhuma mensagem, então $\Pr[C = c \mid M = m]$ não pode depender de m ; caso dependesse, Bayes alteraria a distribuição posterior.

2.2.1 One-Time Pad como exemplo completo

Para mensagens de ℓ bits, escolha $K \xleftarrow{\$} \{0, 1\}^\ell$ e defina

$$C = M \oplus K, \quad M = C \oplus K.$$

Fixe qualquer $m, c \in \{0, 1\}^\ell$. Existe exatamente uma chave capaz de levar m a c , a saber $k = m \oplus c$. Como todas as 2^ℓ chaves são equiprováveis,

$$\Pr[\text{Enc}_K(m) = c] = 2^{-\ell},$$

independentemente de m . Pelo Lema 2.5, o OTP tem sigilo perfeito.

As condições não são opcionais: a chave precisa ser uniforme, tão longa quanto a mensagem, mantida secreta e usada uma única vez. Se a mesma chave cifra m e m' , então

$$c \oplus c' = m \oplus m',$$

e o sigilo perfeito para o conjunto das duas mensagens desaparece.

Para esquemas perfeitamente secretos com espaços finitos e corretude, vale o limite

$$|\mathcal{K}| \geq |\mathcal{M}|.$$

A intuição é que, fixado um cifrado possível c , cada mensagem precisa ter ao menos uma chave que a leve a c ; pela correteza, uma mesma chave não pode decriptar c em duas mensagens diferentes. Logo são necessárias pelo menos tantas chaves quanto mensagens. É esse custo que motiva a relaxação para segurança computacional.

2.2.2 Lema 2.5 versus Definição 2.6

O **Lema 2.5** compara diretamente as distribuições de cifrados produzidas por quaisquer duas mensagens. Nenhum cifrado pode favorecer uma mensagem.

A **Definição 2.6** apresenta o mesmo objetivo como um jogo:

1. o adversário escolhe $m_0, m_1 \in \mathcal{M}$;
2. o oráculo sorteia $b \xleftarrow{\$} \{0, 1\}$;
3. devolve $c \leftarrow \text{Enc}_K(m_b)$;
4. o adversário produz b' .

Sigilo perfeito exige

$$\Pr[b' = b] = \frac{1}{2}$$

até para adversários sem limite computacional.

Na Definição 2.6 do livro, igualdade de tamanho não é escrita separadamente; ela é automática nos exemplos em que $\mathcal{M}$ contém apenas mensagens de um comprimento fixo. Na Definição 3.8, que admite comprimentos variáveis, a restrição $|m_0| = |m_1|$ aparece explicitamente para que o tamanho não entregue o bit b .

Vantagem da formulação por jogo: ela se generaliza facilmente. Basta alterar os poderes do adversário, os oráculos disponíveis e a condição de vitória. É assim que surgem EAV, CPA, CCA, segurança de MACs e segurança de assinaturas.

2.3 Segurança concreta e assintótica

2.3.1 Abordagem concreta

Uma construção pode ser declarada (t, ε) -segura quando nenhum adversário com tempo no máximo t alcança **vantagem** maior que ε . Em um jogo cuja linha de base é $1/2$, isso significa sucesso no máximo $1/2 + \varepsilon$; não significa sucesso no máximo ε .

Essa abordagem responde perguntas de engenharia: quantas operações são necessárias e qual risco residual é aceito?

2.3.2 Abordagem assintótica

O adversário é um algoritmo probabilístico de tempo polinomial, abreviado **PPT**. O parâmetro de segurança é λ , e o sucesso adicional permitido deve ser uma função negligenciável de λ .

Uma função $\mu : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ é **negligenciável** se

$$\forall c > 0 \exists N \forall \lambda \geq N : \quad \mu(\lambda) < \frac{1}{\lambda^c}.$$

Equivalentemente, para todo polinômio positivo p , existe N tal que

$$\lambda \geq N \implies \mu(\lambda) < \frac{1}{p(\lambda)}.$$

Exemplos:

$$2^{-\lambda}, \quad 2^{-\sqrt{\lambda}} \quad \text{são negligenciáveis;}$$

$$\frac{1}{\lambda}, \quad \frac{1}{\lambda^{100}}, \quad \frac{1}{\log \lambda} \quad \text{não são negligenciáveis.}$$

O motivo de $1/\lambda^{100}$ não ser negligenciável é a ordem dos quantificadores: a função deve superar o inverso de **todo** polinômio, não apenas de algum polinômio.

2.3.3 Soma de funções negligenciáveis

Se μ_1 e μ_2 são negligenciáveis, então $\mu_1 + \mu_2$ também é. Fixe $c > 0$. Como cada função é negligenciável, para λ suficientemente grande:

$$\mu_1(\lambda) < \frac{1}{2\lambda^c}, \quad \mu_2(\lambda) < \frac{1}{2\lambda^c}.$$

Logo:

$$\mu_1(\lambda) + \mu_2(\lambda) < \frac{1}{2\lambda^c} + \frac{1}{2\lambda^c} = \frac{1}{\lambda^c}.$$

Também vale: polinômio vezes negligenciável continua negligenciável.

2.4 Definição 3.8 versus Definição 2.6

As duas usam um experimento com duas mensagens. A diferença é o tipo de garantia:

Propriedade	Adversário	Mensagens	Probabilidade de vitória
Def. 2.6: sigilo perfeito	ilimitado	no espaço fixado $\mathcal{M}$	exatamente $1/2$
Def. 3.8: EAV	PPT, recebe 1^λ	mesmo comprimento	no máximo $1/2 + \text{negl}(\lambda)$

A Definição 3.8 introduz o parâmetro de segurança e vê tanto o tempo quanto o sucesso como funções de λ . Ela é estritamente mais fraca: todo esquema perfeitamente secreto é EAV-seguro, mas o pseudo-OTP será EAV-seguro com uma chave menor que a mensagem e, portanto, não poderá ter sigilo perfeito.

A vantagem de $\mathcal{A}$ pode ser escrita como

$$\text{Adv}_{\Pi, \mathcal{A}}^{\text{eav}}(\lambda) = \left| \Pr[\text{PrivK}_{\mathcal{A}, \Pi}^{\text{eav}}(\lambda) = 1] - \frac{1}{2} \right|.$$

Segurança computacional exige vantagem negligenciável para todo adversário PPT.

2.5 Segurança semântica

Intuição: observar o cifrado não deve permitir que um adversário eficiente calcule nenhuma informação sobre a mensagem que ele não conseguiria obter sem o cifrado.

Exemplos de informação proibida:

- um bit específico de m ;
- um predicado $f(m)$;
- se o valor está acima de um limite;
- se duas partes da mensagem são iguais.

Segurança semântica e indistinguibilidade de encriptações são definições equivalentes sob a formulação adequada. Na prova, a explicação importante é:

Se o adversário distingue encriptações de m_0 e m_1 , escolha uma propriedade que tenha valores diferentes nas duas mensagens. Se ele aprende uma propriedade da mensagem, escolha duas distribuições/mensagens nas quais essa propriedade permita distinguir.

2.5.1 O experimento semântico, sem esconder os quantificadores

Considere uma distribuição de mensagens M que possa ser amostrada em tempo polinomial, uma função eficiente f que representa a informação desejada e, opcionalmente, uma função h que representa informação auxiliar já conhecida. O adversário real recebe

$$C \leftarrow \text{Enc}_K(M) \quad \text{e} \quad h(M)$$

e tenta calcular $f(M)$. Segurança semântica afirma que existe um simulador PPT $\mathcal{S}$, que recebe apenas $h(M)$ e o tamanho da mensagem, cujo sucesso fica no máximo uma quantidade negligenciável abaixo do sucesso do adversário real:

$$\Pr[\mathcal{A}(C, h(M)) = f(M)] - \Pr[\mathcal{S}(1^\lambda, |M|, h(M)) = f(M)] \leq \text{negl}(\lambda).$$

O simulador não precisa adivinhar perfeitamente $f(M)$. Ele precisa mostrar que o cifrado não dá uma vantagem significativa sobre aquilo que já seria possível sem vê-lo. Por exemplo, se 90% das mensagens da distribuição começam com zero, adivinhar zero já acerta 90% das vezes; a definição não exige esconder o que a própria distribuição revela.

Essa observação elimina uma confusão comum: “o atacante acertou alguma coisa” não prova insegurança. É necessário comparar o sucesso com uma linha de base sem o cifrado.

2.5.2 Da indistinguibilidade para a proteção de predicados

Suponha que um algoritmo extraia do cifrado um predicado $f(m)$ melhor que a linha de base. Escolha duas mensagens m_0, m_1 para as quais $f(m_0) \neq f(m_1)$. Diante de uma encriptação de uma delas, execute o extrator e use o valor recuperado para adivinhar o bit b . Uma vantagem na extração vira uma vantagem de distinção.

No sentido contrário, se $\mathcal{A}$ distingue encriptações de m_0 e m_1 , defina a informação desejada como “qual das duas mensagens foi escolhida”. O distinguidor já é um algoritmo que aprende essa informação. A prova formal precisa tratar distribuições e informação auxiliar, mas esse é o núcleo lógico da equivalência.

2.6 Geradores pseudoaleatórios

Um PRG é um algoritmo determinístico eficiente

$$G : \{0, 1\}^\lambda \longrightarrow \{0, 1\}^{\ell(\lambda)}, \quad \ell(\lambda) > \lambda.$$

Para semente uniforme $s \xleftarrow{\$} \{0, 1\}^\lambda$, exige-se que $G(s)$ seja computacionalmente indistinguível de $r \xleftarrow{\$} \{0, 1\}^{\ell(\lambda)}$:

$$|\Pr[D(G(s)) = 1] - \Pr[D(r) = 1]| \leq \text{negl}(\lambda)$$

para todo distinguidor PPT D .

2.6.1 As distribuições são realmente diferentes

Se $\ell(\lambda) = 2\lambda$, então:

$$|\text{Im}(G)| \leq 2^\lambda \quad \text{e} \quad |\{0, 1\}^{2\lambda}| = 2^{2\lambda}.$$

A distribuição uniforme cai na imagem de G com probabilidade no máximo

$$\frac{2^\lambda}{2^{2\lambda}} = 2^{-\lambda}.$$

Já uma saída de G cai nessa imagem com probabilidade 1. Logo a distância estatística entre as distribuições é pelo menos $1 - 2^{-\lambda}$, isto é, quase máxima.

Um algoritmo ilimitado poderia enumerar todas as sementes e distinguir quase sempre. A segurança afirma somente que nenhum algoritmo **PPT** consegue fazer esse teste. Portanto, as distribuições são estatisticamente muito diferentes, embora computacionalmente indistinguíveis.

2.7 Pseudo-One-Time Pad

Com $|m| = \ell(\lambda)$:

$$\text{Enc}_k(m) = m \oplus G(k), \quad \text{Dec}_k(c) = c \oplus G(k).$$

A chave tem apenas λ bits, mas cifra mensagens maiores. Não há contradição com o limite do OTP: o pseudo-OTP oferece segurança computacional, não sigilo perfeito.

Essa construção é segura no experimento EAV de **uma única encriptação**. Reutilizar a mesma chave em duas mensagens reutiliza o pad $G(k)$ e revela o XOR dos textos claros. Para várias mensagens, é necessário introduzir aleatoriedade, estado ou nonce de modo que pads diferentes sejam produzidos.

2.8 Provas por redução

Uma redução tem a forma:

$$\text{adversário } \mathcal{A} \text{ quebra } \Pi \implies \text{algoritmo } \mathcal{B}^{\mathcal{A}} \text{ quebra a hipótese } H.$$

Estrutura obrigatória:

1. **Suposição:** existe $\mathcal{A}$ com vantagem não negligenciável contra Π .
2. **Construção:** $\mathcal{B}$ recebe uma instância do problema H e usa $\mathcal{A}$ como sub-rotina.
3. **Simulação:** a visão fornecida a $\mathcal{A}$ deve ter a distribuição correta.
4. **Análise de casos:** quando a entrada de $\mathcal{B}$ é real e quando é aleatória/falsa.
5. **Relação de vantagens:** a vantagem de $\mathcal{B}$ é calculada a partir da vantagem de $\mathcal{A}$.
6. **Contradição:** isso viola a hipótese de segurança de H .

No pseudo-OTP, o redutor recebe w , que é $G(k)$ ou uniforme. Ele escolhe b , entrega $c = w \oplus m_b$ ao adversário e responde “PRG” quando $b' = b$. Se w é uniforme, o jogo é OTP; se $w = G(k)$, o jogo é o esquema real.

2.8.1 Redução do pseudo-OTP, com a vantagem calculada

Seja $\mathcal{A}$ um adversário EAV contra o pseudo-OTP. O distinguidor $\mathcal{B}$ para o PRG recebe uma cadeia $w \in \{0, 1\}^{\ell(\lambda)}$ e executa:

1. obtém de $\mathcal{A}$ duas mensagens m_0, m_1 de mesmo comprimento;
2. sorteia $b \xleftarrow{\$} \{0, 1\}$;
3. forma $c = m_b \oplus w$ e entrega c a $\mathcal{A}$;
4. recebe b' e devolve 1 se, e somente se, $b' = b$.

Há dois mundos.

Mundo PRG: se $w = G(k)$, a simulação é exatamente a encriptação real. Portanto,

$$\Pr[\mathcal{B}(G(k)) = 1] = \Pr[b' = b \mid \text{pseudo-OTP}].$$

Mundo uniforme: se w é uniforme, $m_b \oplus w$ também é uniforme e independe de b . Isso é um OTP novo para uma única mensagem, logo

$$\Pr[\mathcal{B}(w) = 1 \mid w \leftarrow U_\ell] = \frac{1}{2}.$$

Consequentemente,

$$\text{Adv}_{G, \mathcal{B}}^{\text{prg}} = \left| \Pr[b' = b \mid \text{pseudo-OTP}] - \frac{1}{2} \right| = \text{Adv}_{\Pi, \mathcal{A}}^{\text{eav}}.$$

Se a vantagem de $\mathcal{A}$ fosse não negligenciável, a vantagem de $\mathcal{B}$ também seria, contradizendo a segurança do PRG. Note os três detalhes que tornam a prova válida: $\mathcal{B}$ é eficiente se $\mathcal{A}$ é eficiente; a simulação do mundo real é perfeita; e no mundo uniforme a probabilidade é exatamente $1/2$.

2.8.2 Híbridos

Quando uma construção usa muitos objetos pseudoaleatórios, costuma-se trocar um objeto por vez. Se H_0 é o jogo real, H_t o jogo ideal e $H_1, \dots, H_{t-1}$ são híbridos, então

$$|\Pr[H_0 = 1] - \Pr[H_t = 1]| \leq \sum_{i=0}^{t-1} |\Pr[H_i = 1] - \Pr[H_{i+1} = 1]|.$$

Se a diferença total é não negligenciável e t é polinomial, pelo menos uma troca adjacente tem diferença não negligenciável. Essa troca aponta exatamente onde embutir o desafio da redução.

2.9 EAV e CPA

2.9.1 EAV: uma observação passiva

O adversário escolhe m_0, m_1 , recebe uma encriptação de m_b e tenta descobrir b .

2.9.2 CPA: texto claro escolhido

Além do desafio, o adversário consulta um oráculo de encriptação:

$$m \mapsto \text{Enc}_k(m).$$

Uma cifra determinística perde imediatamente: o adversário pede $\text{Enc}_k(m_0)$ e compara com o desafio.

O jogo completo possui duas fases de consulta:

1. $k \leftarrow \text{Gen}(1^\lambda)$;
2. $\mathcal{A}$ consulta $\text{Enc}_k(\cdot)$ quantas vezes polinomiais quiser;
3. $\mathcal{A}$ envia m_0, m_1 com $|m_0| = |m_1|$;
4. o desafiante sorteia $b \leftarrow \{0, 1\}$ e devolve $c^* \leftarrow \text{Enc}_k(m_b)$;
5. $\mathcal{A}$ continua consultando o oráculo e, ao final, devolve b' .

A vantagem é

$$\text{Adv}_{\Pi, \mathcal{A}}^{\text{cpa}}(\lambda) = \left| \Pr[b' = b] - \frac{1}{2} \right|.$$

A igualdade de comprimentos impede uma vitória trivial pelo tamanho do cifrado. A segunda fase impede uma prova que só funcione quando o atacante para de interagir após o desafio.

2.9.3 Ataque explícito a uma cifra determinística

Escolha $m_0 \neq m_1$ de mesmo tamanho. Antes do desafio, consulte

$$c_0 \leftarrow \text{Enc}_k(m_0).$$

Ao receber $c^* = \text{Enc}_k(m_b)$, responda $b' = 0$ se $c^* = c_0$ e $b' = 1$ caso contrário. Como a encriptação é determinística e correta, cifrar mensagens distintas não pode produzir o mesmo cifrado: se produzisse, a decifração do mesmo valor precisaria devolver duas mensagens. Logo o ataque acerta sempre:

$$\Pr[b' = b] = 1, \quad \text{Adv}^{\text{cpa}} = \frac{1}{2}.$$

2.9.4 PRF e PRP: a abstração por trás dos modos

Uma família de funções pseudoaleatórias é indexada por uma chave:

$$F_k : \{0, 1\}^n \rightarrow \{0, 1\}^m.$$

O adversário, com acesso de oráculo, não distingue F_k de uma função verdadeiramente aleatória escolhida entre todas as funções com esse domínio e contradomínio. Uma função aleatória pode produzir colisões.

Uma permutação pseudoaleatória tem domínio e contradomínio iguais e cada P_k é bijetiva:

$$P_k : \{0, 1\}^n \rightarrow \{0, 1\}^n.$$

Ela deve ser indistinguível de uma permutação uniforme. Cifras de bloco como AES são modeladas como PRPs; para cada chave, a decifração é a permutação inversa. A diferença é observável após consultas suficientes: uma função aleatória pode colidir, uma permutação nunca. Para número de consultas muito menor que $2^{n/2}$, essa diferença é pequena, o que explica por que uma PRP frequentemente pode substituir uma PRF em análises com uma perda do tipo aniversário.

Consequências:

- CPA exige aleatoriedade, nonce único ou estado seguro;
- mensagens do desafio devem ter o mesmo tamanho;
- o adversário pode escolher as consultas adaptativamente;
- confidencialidade CPA não fornece, por si só, integridade.

3 Parte II — Criptografia simétrica

3.1 Cifra de fluxo versus cifra de bloco

Uma **cifra de fluxo** produz um keystream pseudoaleatório:

$$z_1 z_2 \dots := G(k, \text{nonce}), \quad c = m \oplus z.$$

Reutilizar o mesmo keystream em duas mensagens produz

$$c \oplus c' = (m \oplus z) \oplus (m' \oplus z) = m \oplus m',$$

eliminando a chave do cálculo.

Uma **cifra de bloco** é uma família de permutações indexadas por chave:

$$E : \mathcal{K} \times \{0, 1\}^n \rightarrow \{0, 1\}^n,$$

com inversa eficiente $D_k = E_k^{-1}$. Idealmente, E_k se comporta como uma permutação aleatória para quem não conhece k : uma PRP.

Conceito	Entrada/saída	Propriedade central
PRG	semente curta $\rightarrow$ string longa	saída parece uniforme
PRF	entrada arbitrária fixa $\rightarrow$ saída	função parece aleatória
PRP/block cipher	bloco $\rightarrow$ bloco do mesmo tamanho	permutação parece aleatória

3.2 Rede de Feistel

Divida o bloco em (L_0, R_0) . Na rodada i :

$$L_i = R_{i-1}, \quad R_i = L_{i-1} \oplus F_i(R_{i-1}).$$

A função F_i não precisa ser invertível. Para desfazer a rodada:

$$R_{i-1} = L_i, \quad L_{i-1} = R_i \oplus F_i(L_i).$$

Isso funciona porque $x \oplus y \oplus y = x$. Uma rede de quatro rodadas usa as subchaves k_1, k_2, k_3, k_4 na ida e k_4, k_3, k_2, k_1 na volta, mantendo a mesma estrutura.

O que explicar em prova: Feistel cria uma permutação invertível mesmo quando a função de rodada não é invertível.

3.2.1 Exemplo completo com quatro rodadas

Use metades de 4 bits e, apenas para tornar a conta visível, defina

$$F_{k_i}(R) = R \oplus k_i.$$

Tome

$$(L_0, R_0) = (\mathbf{A}, 3), \quad (k_1, k_2, k_3, k_4) = (5, \mathbf{C}, 6, 9).$$

Cada linha aplica $L_i = R_{i-1}$ e $R_i = L_{i-1} \oplus (R_{i-1} \oplus k_i)$:

i	L_i	R_i	cálculo de R_i
0	A	3	entrada
1	3	C	$\mathbf{A} \oplus (3 \oplus 5) = \mathbf{C}$
2	C	3	$3 \oplus (\mathbf{C} \oplus \mathbf{C}) = 3$
3	3	9	$\mathbf{C} \oplus (3 \oplus 6) = 9$
4	9	3	$3 \oplus (9 \oplus 9) = 3$

O resultado é $(L_4, R_4) = (9, 3)$. Para voltar uma rodada, não se calcula F^{-1} . Usa-se

$$R_{i-1} = L_i, \quad L_{i-1} = R_i \oplus F_{k_i}(L_i).$$

Da saída, com $k_4 = 9$:

$$R_3 = 9, \quad L_3 = 3 \oplus (9 \oplus 9) = 3.$$

Repetindo com k_3, k_2, k_1 , recupera-se $(\mathbf{A}, 3)$. O exemplo também mostra por que a ordem das subchaves precisa ser invertida.

Poucas rodadas não bastam para segurança. Em uma rodada, a metade esquerda da saída é literalmente a metade direita da entrada; em duas rodadas ainda há relações simples entre entrada e saída. O resultado de Luby–Rackoff dá o fundamento teórico: três rodadas com PRFs independentes constroem uma PRP e quatro constroem uma strong PRP, sob as definições apropriadas. Isso não quer dizer que uma cifra prática deva usar apenas quatro rodadas; DES usa dezesseis porque funções reais, margens de segurança e ataques criptanalíticos exigem mais cuidado.

3.3 DES e TripleDES

DES:

- bloco de 64 bits;
- chave nominal de 64 bits, mas apenas 56 efetivos;
- rede de Feistel com 16 rodadas;
- principal fraqueza prática: busca exaustiva em 2^{56} ;
- bloco de 64 bits também é pequeno para grandes volumes de dados.

DES continua importante para compreender a história do projeto de cifras: foi o primeiro padrão civil amplamente adotado, popularizou redes de Feistel, motivou o desenvolvimento público de criptoanálise diferencial e linear e resistiu a décadas de análise estrutural. Sua obsolescência prática não torna sua arquitetura irrelevante;

ela mostra, sobretudo, que uma cifra pode ter estrutura bem estudada e ainda assim falhar porque a chave e o bloco ficaram pequenos.

TripleDES aplica DES três vezes, tipicamente em EDE:

$$c = E_{k_3}(D_{k_2}(E_{k_1}(m))).$$

EDE preservou compatibilidade com DES quando as três chaves coincidiam. 3DES aumentou a segurança, mas é lento e mantém o bloco pequeno. Sua importância é histórica: mostrou como prolongar a vida de um primitivo sem redesenhar toda a infraestrutura.

3.3.1 Por que 3DES não entrega simplesmente 168 bits

Com duas chaves, a construção EDE é

$$C = E_{k_1}(D_{k_2}(E_{k_1}(M))).$$

Com três chaves há 168 bits nominais, mas ataques de *meet-in-the-middle* e suas variantes reduzem a margem efetiva. A ideia de *meet-in-the-middle* para uma dupla encriptação $C = E_{k_2}(E_{k_1}(M))$ é:

1. calcule e armazene $E_{k_1}(M)$ para todas as chaves k_1 ;
2. calcule $D_{k_2}(C)$ para todas as chaves k_2 ;
3. procure valores intermediários iguais;
4. confirme candidatos com outros pares mensagem-cifrado.

O tempo fica da ordem de 2^k , e não 2^{2k} , com grande memória. A estrutura tripla dificulta esse ataque direto, mas não remove todas as reduções genéricas. Além disso, o bloco de apenas 64 bits impõe limites de volume pelo efeito de aniversário. Portanto, “aplicar DES três vezes” não resolve o problema tão bem quanto adotar uma cifra moderna com bloco de 128 bits.

Como ordem de grandeza, 3DES com três chaves é tratado como oferecendo cerca de 112 bits de segurança clássica, não 168 bits completos.

3.4 AES

AES é uma rede de substituição-permutação, não uma rede de Feistel. O NIST conduziu uma competição pública internacional: recebeu quinze candidatos, selecionou cinco finalistas após análise aberta e escolheu Rijndael em 2000. A escolha considerou segurança, desempenho em software e hardware, flexibilidade e simplicidade de implementação; não foi apenas uma comparação de tamanhos de chave.

Todas as variantes usam bloco de 128 bits:

Variante	Chave	Rodadas
AES-128	128 bits	10
AES-192	192 bits	12
AES-256	256 bits	14

O estado é uma matriz 4×4 de bytes. As transformações são:

1. **SubBytes:** aplica uma S-box bijetiva a cada byte;
2. **ShiftRows:** desloca ciclicamente as linhas;
3. **MixColumns:** aplica uma transformação linear invertível em cada coluna, sobre $GF(2^8)$;
4. **AddRoundKey:** XOR com a subchave da rodada.

Há um AddRoundKey inicial; a última rodada omite MixColumns.

3.4.1 Por que AES pode ser decriptado?

- SubBytes é invertível porque sua S-box é uma permutação;
- ShiftRows é invertido por deslocamentos na direção oposta;
- MixColumns usa uma matriz invertível em $GF(2^8)$;
- AddRoundKey é sua própria inversa:

$$(x \oplus k) \oplus k = x;$$

- o key schedule permite reconstruir/gerar as subchaves na ordem necessária.

A decrptação aplica as inversas na ordem reversa. AES não precisa ser Feistel porque **cada transformação individual** é invertível.

Com a chave original, o algoritmo de expansão gera $N_r + 1$ subchaves. A deciptação começa pela última e termina pela primeira. No *Inverse Cipher*, a ordem conceitual é

InvShiftRows, InvSubBytes, AddRoundKey, InvMixColumns.

No *Equivalent Inverse Cipher* de FIPS 197, pode-se reorganizar a rodada para ter aparência semelhante à encriptação, aplicando InvMixColumns também às subchaves internas. As duas descrições calculam a mesma permutação inversa. O ponto essencial é que as subchaves não precisam ser “adivinhadas”: são derivadas deterministicamente da chave e usadas em ordem reversa.

3.4.2 Anatomia de uma rodada do AES

Os 16 bytes de entrada preenchem o estado por colunas:

$$S_{r,c} = \text{entrada}_{4c+r}, \quad 0 \leq r, c < 4.$$

Isso importa ao fazer uma conta: os quatro primeiros bytes formam a primeira coluna, não a primeira linha.

SubBytes. A S-box não é uma tabela escolhida ao acaso. Para byte não nulo, calcula-se o inverso multiplicativo em $GF(2^8)$, usando o módulo

$$x^8 + x^4 + x^3 + x + 1,$$

e depois uma transformação afim sobre os bits. O zero recebe inverso convencional zero antes da transformação afim. Por exemplo, a S-box leva 53 a ED. A transformação afim evita estruturas algébricas ruins que permaneceriam se fosse usado apenas o inverso.

ShiftRows. As linhas 0, 1, 2, 3 são deslocadas respectivamente por 0, 1, 2, 3 posições. Assim, bytes que estavam na mesma coluna são espalhados por colunas diferentes antes da próxima mistura.

MixColumns. Cada coluna é multiplicada pela matriz

$$\begin{pmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{pmatrix}$$

em $GF(2^8)$. O exemplo clássico

$$\begin{pmatrix} \text{D4} \\ \text{BF} \\ \text{5D} \\ \text{30} \end{pmatrix} \mapsto \begin{pmatrix} \text{04} \\ \text{66} \\ \text{81} \\ \text{E5} \end{pmatrix}$$

mostra que cada byte de saída depende dos quatro bytes da coluna. A multiplicação por 02 é um deslocamento à esquerda seguido de XOR com 1B quando o bit mais significativo era 1; multiplicar por 03 é multiplicar por 02 e fazer XOR com o byte original.

AddRoundKey. O estado sofre XOR byte a byte com 128 bits do key schedule. É aqui que entra a chave; as outras três operações são públicas.

SubBytes fornece não linearidade; ShiftRows e MixColumns fornecem difusão; AddRoundKey injeta o segredo. Depois de várias rodadas, alterar um bit da entrada ou da chave afeta muitos bits da saída.

3.4.3 Key schedule e tamanhos que não devem ser confundidos

O bloco do AES é sempre 128 bits. A chave pode ter 128, 192 ou 256 bits, e essa escolha altera o número de rodadas. “AES-256” não significa bloco de 256 bits. O key schedule expande a chave em uma subchave de 128 bits para cada AddRoundKey, usando rotação de palavras, aplicação da S-box e constantes de rodada. Ele não é uma simples repetição da chave.

3.5 Corpos finitos binários

Um bitstring $b_{m-1} \dots b_1 b_0$ representa o polinômio

$$b_{m-1}x^{m-1} + \dots + b_1x + b_0$$

com coeficientes em $GF(2)$. Soma e subtração são XOR. Em

$$GF(2^m) \cong GF(2)[x]/(f(x)),$$

$f(x)$ deve ser irredutível de grau m . Multiplica-se os polinômios e reduz-se o resultado módulo $f(x)$.

3.5.1 Exemplo em $GF(2^4)$

Com

$$f(x) = x^4 + x + 1,$$

temos

$$0101 = x^2 + 1, \quad 1101 = x^3 + x^2 + 1.$$

O produto antes da redução é

$$(x^2 + 1)(x^3 + x^2 + 1) = x^5 + x^4 + x^3 + 1.$$

Reduzindo módulo $x^4 + x + 1$:

$$x^5 + x^4 + x^3 + 1 \equiv x^3 + x^2,$$

portanto

$$0101 \cdot 1101 = 1100.$$

Também:

$$(1010)^{-1} = 1100,$$

pois $(x^3 + x)(x^3 + x^2) \equiv 1 \pmod{x^4 + x + 1}$.

Para conferir a inversão, multiplique:

$$(x^3 + x)(x^3 + x^2) = x^6 + x^5 + x^4 + x^3.$$

Do módulo $x^4 + x + 1 = 0$ vem $x^4 \equiv x + 1$. Multiplicando essa relação por x e x^2 :

$$x^5 \equiv x^2 + x, \quad x^6 \equiv x^3 + x^2.$$

Logo

$$(x^3 + x^2) + (x^2 + x) + (x + 1) + x^3 = 1,$$

pois termos repetidos se cancelam em característica 2. O algoritmo de Euclides estendido também poderia encontrar o inverso diretamente, trocando inteiros por polinômios sobre $GF(2)$.

3.5.2 Multiplicação em GCM

GCM trabalha em $GF(2^{128})$. Uma tabela de índices teria tamanho da ordem de 2^{128} e é inviável. A multiplicação é implementada por operações de shift e XOR, com redução pelo polinômio fixado pelo padrão:

$$x^{128} + x^7 + x^2 + x + 1.$$

Uma forma de executar o produto é o algoritmo *shift-and-XOR*. Inicialize $Z = 0$ e $V = X$. Percorra os 128 bits de Y ; quando o bit atual vale 1, faça $Z \leftarrow Z \oplus V$. Em seguida desloque V uma posição. Se o bit que saiu exigir redução, faça XOR com a representação do polinômio redutor. Na convenção de bits usada por GCM, isso aparece como

$$R = \text{E100} \cdots 00$$

e a atualização é

$$V \leftarrow \begin{cases} V \gg 1, & \text{lsb}(V) = 0, \\ (V \gg 1) \oplus R, & \text{lsb}(V) = 1. \end{cases}$$

Após 128 passos, $Z = X \cdot Y$. O custo é linear em 128, enquanto uma tabela de log/antilog teria ordem de 2^{128} entradas. É importante fixar a convenção de ordem dos bits: deslocar para o outro lado é possível, mas muda a constante que representa a redução.

3.6 Modos de operação

Considere

$$m = m_1 \| m_2 \| \dots \| m_\ell.$$

3.6.1 ECB

$$c_i = E_k(m_i), \quad m_i = D_k(c_i).$$

Blocos iguais produzem cifrados iguais. ECB é determinístico, revela padrões e não é CPA-seguro. Não deve ser usado para mensagens estruturadas.

3.6.2 CBC

Escolha $c_0 = IV$ uniforme e imprevisível:

$$c_i = E_k(m_i \oplus c_{i-1}).$$

Decriptação:

$$m_i = D_k(c_i) \oplus c_{i-1}.$$

O IV é transmitido com o cifrado; não precisa ser secreto. Para a formulação clássica de segurança CPA, deve ser novo e imprevisível.

3.6.2.1 Ida, volta e padding Para dois blocos:

$$c_1 = E_k(m_1 \oplus IV), \quad c_2 = E_k(m_2 \oplus c_1).$$

Aplicar a inversa recupera

$$D_k(c_1) \oplus IV = (m_1 \oplus IV) \oplus IV = m_1,$$

$$D_k(c_2) \oplus c_1 = (m_2 \oplus c_1) \oplus c_1 = m_2.$$

CBC precisa de padding quando o comprimento não é múltiplo do bloco. O padding deve ser não ambíguo e sua validação não pode revelar, por mensagens ou tempo, por que um cifrado foi rejeitado; respostas distinguíveis podem criar um *padding oracle*. Isso reforça um ponto central: CBC sozinho fornece confidencialidade, não autenticidade.

Se um atacante inverte um bit de c_i , o bloco de texto m_i fica embaralhado e o mesmo bit de m_{i+1} é invertido de forma controlada, pois

$$m_{i+1} = D_k(c_{i+1}) \oplus c_i.$$

Essa maleabilidade existe mesmo que o atacante não conheça a chave.

3.6.3 Chained CBC

Chained CBC usa o último bloco cifrado da mensagem anterior como IV da próxima. O valor fica conhecido antes de o adversário escolher a próxima mensagem. Isso permite consultas construídas para testar relações entre blocos e quebra CPA. “Parecer equivalente a concatenar mensagens” não basta para garantir segurança no modelo adaptativo.

3.6.3.1 Ataque explícito ao IV previsível Suponha que uma consulta use IV V para cifrar o bloco X e produza

$$C = E_k(X \oplus V).$$

No Chained CBC, C será o IV público da próxima mensagem. Escolha no desafio

$$M_0 = X \oplus V \oplus C$$

e qualquer $M_1 \neq M_0$. Se M_0 for cifrada, o primeiro bloco será

$$C^* = E_k(M_0 \oplus C) = E_k(X \oplus V) = C.$$

Assim, responda $b' = 0$ quando $C^* = C$ e $b' = 1$ caso contrário. Para $M_1 \neq M_0$, a bijetividade de E_k impede igualdade no outro caso. O adversário acerta sempre. O IV de CBC pode ser público depois de escolhido; o que esse ataque explora é conhecê-lo **antes** de escolher a mensagem.

3.6.4 CTR

Para nonce N e contador i :

$$z_i = E_k(N \parallel \langle i \rangle), \quad c_i = m_i \oplus z_i.$$

Decifração usa a mesma operação:

$$m_i = c_i \oplus z_i.$$

Vantagens:

- paralelizável;
- acesso aleatório a blocos;
- não exige padding;
- usa apenas a direção de encriptação de AES.

Regra crítica: o par chave–nonce nunca pode repetir. Repetição reutiliza o keystream.

Se duas mensagens usam o mesmo nonce,

$$c \oplus c' = m \oplus m'.$$

Se m for conhecido, a outra mensagem é recuperada por

$$m' = c \oplus c' \oplus m.$$

Também é possível inverter um bit escolhido do texto claro invertendo o mesmo bit do cifrado. CTR localiza erros, mas é maleável e precisa ser combinado com autenticação.

3.6.5 GCM

GCM combina:

- CTR para confidencialidade;
- GHASH para autenticar cifrado e dados associados.

Defina

$$H = E_k(0^{128})$$

e use multiplicação em $GF(2^{128})$ para acumular os blocos de AAD e cifrado. Em alto nível:

$$T = \text{MSB}_t(E_k(J_0) \oplus \text{GHASH}_H(A, C)).$$

O receptor recalcula T e aceita somente se a comparação for válida. GCM é AEAD: protege confidencialidade e integridade, incluindo dados associados A que não são cifrados. Reutilizar nonce em GCM é catastrófico: além de repetir o keystream de CTR, cria relações que comprometem a autenticação.

3.6.6 Verificação e estrutura de GHASH

De forma esquemática, GHASH acumula

$$A_1, \dots, A_u, C_1, \dots, C_v, \text{len}(A) \parallel \text{len}(C)$$

como uma avaliação polinomial em H . O bloco final vincula os comprimentos e evita ambiguidades de concatenação. Os dados associados podem ser cabeçalhos que devem ficar visíveis, mas não podem ser alterados.

Na recepção:

1. derive $H = E_k(0^{128})$ e J_0 a partir do nonce;
2. recalcule $S = \text{GHASH}_H(A, C)$;

3. calcule $T' = \text{MSB}_t(E_k(J_0) \oplus S)$;
4. compare T' e a tag recebida sem vazamento de tempo;
5. aceite apenas se forem iguais; então recupere os blocos por CTR, $m_i = c_i \oplus E_k(\text{inc}^i(J_0))$.

Em uma interface segura, texto claro não autenticado não é entregue à aplicação. Alterar nonce, AAD, cifrado, tag ou comprimentos deve levar à rejeição, exceto com a pequena probabilidade de falsificação permitida pelo tamanho da tag e pelos limites de uso.

3.6.7 Propagação de erro e maleabilidade

Modo	Efeito de um bit invertido no cifrado	Sem autenticação, o atacante pode alterar?
ECB	embaralha o bloco correspondente	sim
CBC	embaralha m_i e inverte o bit correspondente de m_{i+1}	sim
CTR	inverte exatamente o mesmo bit de m_i	sim
GCM	a tag deve falhar e a mensagem deve ser rejeitada	falsificação deve ser inviável

3.7 Resumo comparativo dos modos

Modo	Aleatoriedade ou nonce	Paraleliza encriptação?	Autentica?	Problema principal
ECB	não	sim	não	revela padrões
CBC	IV novo	não	não	maleável; exige padding
CTR	nonce único	sim	não	reutilização do nonce
GCM	nonce único	sim	sim	reutilização do nonce

4 Parte III — Aritmética modular, grupos e primos

4.1 Divisibilidade, máximo divisor comum e Bézout

Para inteiros a, b com $b \neq 0$, a divisão euclidiana escreve

$$a = qb + r, \quad 0 \leq r < |b|.$$

O algoritmo de Euclides usa repetidamente

$$\gcd(a, b) = \gcd(b, a \bmod b)$$

até o resto ser zero.

O algoritmo de Euclides **estendido** encontra x, y tais que

$$ax + by = \gcd(a, b).$$

Durante o algoritmo de Euclides, cada resto é uma combinação linear dos dois números anteriores. Guardando os coeficientes dessas combinações, obtém-se Bézout sem precisar refazer toda a conta. Uma forma iterativa mantém

$$r_i = s_i a + t_i b.$$

Comece com

$$(r_0, s_0, t_0) = (a, 1, 0), \quad (r_1, s_1, t_1) = (b, 0, 1).$$

Para $q_i = \lfloor r_{i-1}/r_i \rfloor$, atualize

$$(r_{i+1}, s_{i+1}, t_{i+1}) = (r_{i-1}, s_{i-1}, t_{i-1}) - q_i(r_i, s_i, t_i).$$

O último resto não nulo é o MDC, e os coeficientes da mesma linha são os x, y procurados.

Quando $\gcd(a, n) = 1$, a identidade

$$ax + ny = 1$$

implica

$$ax \equiv 1 \pmod{n},$$

portanto $x = a^{-1} \pmod{n}$.

4.1.1 Exemplo: inverso de 17 módulo 101

$$101 = 5 \cdot 17 + 16, \quad 17 = 1 \cdot 16 + 1.$$

Voltando:

$$1 = 17 - 16 = 17 - (101 - 5 \cdot 17) = 6 \cdot 17 - 101.$$

Logo:

$$17^{-1} \equiv 6 \pmod{101}.$$

Na identidade pedida, os coeficientes são explicitamente

$$x = 6, \quad y = -1, \quad 17x + 101y = 1.$$

4.2 Aritmética modular

Escrevemos

$$a \equiv b \pmod{n}$$

quando $n \mid (a - b)$. Congruências podem ser somadas e multiplicadas:

$$a \equiv b \pmod{n}, \quad c \equiv d \pmod{n} \implies \begin{cases} a + c \equiv b + d \pmod{n}, \\ ac \equiv bd \pmod{n}. \end{cases}$$

Divisão não é uma operação livre. Só se pode multiplicar por a^{-1} quando a é invertível módulo n , isto é, quando $\gcd(a, n) = 1$.

Por exemplo,

$$2x \equiv 2 \pmod{6}$$

não permite concluir simplesmente $x \equiv 1 \pmod{6}$: tanto $x = 1$ quanto $x = 4$ satisfazem a congruência. Cancelar um fator não invertível perde soluções ou exige também ajustar o módulo.

4.3 Square-and-Multiply

Para calcular $a^e \pmod{n}$, escreva e em binário e faça quadraturas sucessivas, reduzindo módulo n a cada passo.

Exemplo:

$$13 = (1101)_2 = 8 + 4 + 1.$$

Para $3^{13} \pmod{17}$:

$$3^1 \equiv 3,$$

$$3^2 \equiv 9,$$

$$3^4 \equiv 9^2 \equiv 13,$$

$$3^8 \equiv 13^2 \equiv 16 \pmod{17}.$$

Assim:

$$3^{13} \equiv 3^8 \cdot 3^4 \cdot 3 \equiv 16 \cdot 13 \cdot 3 \equiv 12 \pmod{17}.$$

O número de multiplicações é $O(\log e)$, não $O(e)$.

No algoritmo da esquerda para a direita, inicie $r = 1$ e percorra os bits do expoente do mais significativo ao menos significativo. Para cada bit, faça

$$r \leftarrow r^2 \bmod n;$$

se o bit vale 1, faça também

$$r \leftarrow r \cdot a \bmod n.$$

Essa descrição deixa claro por que o custo cresce com o número de bits do expoente. Em implementações criptográficas, ramificações e acessos à memória dependentes de bits secretos precisam ser evitados para não criar canais laterais; isso é uma questão de implementação, separada da correção matemática do algoritmo.

4.4 Grupos

Um grupo $(G, \circ)$ satisfaz:

1. fechamento: $a \circ b \in G$;
2. associatividade;
3. existência de identidade e ;
4. existência de inverso para cada elemento.

Se a operação é comutativa, o grupo é abeliano.

4.4.1 Grupos aditivos e multiplicativos

$$\mathbb{Z}_n^+ = \{0, 1, \dots, n-1\}$$

usa soma módulo n e tem n elementos.

$$\mathbb{Z}_n^* = \{a \in \{1, \dots, n-1\} : \gcd(a, n) = 1\}$$

usa multiplicação módulo n e tem

$$|\mathbb{Z}_n^*| = \varphi(n)$$

elementos.

4.4.2 Ordem de grupo e ordem de elemento

A ordem do grupo é $|G|$. A ordem de $g \in G$ é o menor inteiro positivo r tal que

$$g^r = e.$$

Pelo teorema de Lagrange:

$$\text{ord}(g) \mid |G|.$$

No grupo aditivo $\mathbb{Z}_n^+$:

$$\text{ord}(a) = \frac{n}{\gcd(a, n)}.$$

4.4.3 Grupo cíclico e gerador

G é cíclico se existe $g \in G$ tal que

$$G = \langle g \rangle = \{g^0, g^1, \dots, g^{|G|-1}\}.$$

Nesse caso, g é um gerador e

$$\text{ord}(g) = |G|.$$

Se a ordem de G é um primo q , todo elemento diferente da identidade tem ordem q e é gerador.

Se a fatoração de $|G| = N$ é conhecida, há um teste eficiente para gerador. Sejam $r_1, \dots, r_t$ os fatores **primos distintos** de N . Então

$$g \text{ gera } G \iff g^{N/r_i} \neq e \text{ para todo } i.$$

Se algum teste der a identidade, a ordem de g divide N/r_i e não pode ser N . Se nenhum der, nenhum fator primo pode ser removido de N , portanto a ordem é N .

4.5 Função de Euler

$$\varphi(n) = |\mathbb{Z}_n^*|.$$

Se

$$n = \prod_{i=1}^t p_i^{e_i},$$

então

$$\varphi(n) = n \prod_{i=1}^t \left(1 - \frac{1}{p_i}\right).$$

Casos importantes:

$$\varphi(p) = p - 1$$

para p primo, e

$$\varphi(pq) = (p - 1)(q - 1)$$

para primos distintos p, q .

Exemplos:

$$\varphi(100) = 40, \quad \varphi(101) = 100, \quad \varphi(1001) = 720,$$

pois $1001 = 7 \cdot 11 \cdot 13$.

4.6 Teoremas de Fermat e Euler

Se p é primo e $a \not\equiv 0 \pmod{p}$:

$$a^{p-1} \equiv 1 \pmod{p}.$$

Mais geralmente, se $\gcd(a, n) = 1$:

$$a^{\varphi(n)} \equiv 1 \pmod{n}.$$

4.6.1 Uso para testar composição

Se

$$a^{n-1} \not\equiv 1 \pmod{n},$$

então n é composto.

A recíproca é falsa: alguns compostos passam para certas bases (pseudoprimos), e números de Carmichael passam no teste de Fermat para toda base coprima. Portanto, Fermat pode provar composição quando encontra uma testemunha, mas sozinho não é um bom teste geral de primalidade.

4.7 Teorema Chinês do Resto

Se $\gcd(n_1, n_2) = 1$, o mapa

$$x \bmod (n_1 n_2) \mapsto (x \bmod n_1, x \bmod n_2)$$

é uma bijeção compatível com soma e multiplicação:

$$\mathbb{Z}_{n_1 n_2} \cong \mathbb{Z}_{n_1} \times \mathbb{Z}_{n_2}.$$

Para $n = pq$ com primos distintos:

$$\mathbb{Z}_n^* \cong \mathbb{Z}_p^* \times \mathbb{Z}_q^*.$$

Essa decomposição explica:

- a fórmula $\varphi(pq) = (p-1)(q-1)$;
- a prova completa de corretude do RSA;
- otimizações de deciptação RSA;
- por que conhecer p, q facilita operações módulo n .

4.7.1 Reconstrução explícita por CRT

Para módulos coprimos $n_1, \dots, n_t$, defina

$$N = \prod_i n_i, \quad N_i = \frac{N}{n_i}, \quad u_i = N_i^{-1} \pmod{n_i}.$$

A solução de $x \equiv a_i \pmod{n_i}$ para todo i é

$$x \equiv \sum_i a_i N_i u_i \pmod{N}.$$

Exemplo:

$$x \equiv 2 \pmod{3}, \quad x \equiv 3 \pmod{5}.$$

Temos $N = 15$, $N_1 = 5$, $5^{-1} \equiv 2 \pmod{3}$, $N_2 = 3$ e $3^{-1} \equiv 2 \pmod{5}$. Logo

$$x \equiv 2 \cdot 5 \cdot 2 + 3 \cdot 3 \cdot 2 = 38 \equiv 8 \pmod{15}.$$

De fato, $8 \pmod{3} = 2$ e $8 \pmod{5} = 3$. Dizer $\mathbb{Z}_{pq} \cong \mathbb{Z}_p \times \mathbb{Z}_q$ significa mais que haver a mesma quantidade de elementos: soma e multiplicação módulo pq correspondem a fazer as operações separadamente nos dois componentes. Restringindo aos elementos invertíveis, obtém-se $\mathbb{Z}_{pq}^* \cong \mathbb{Z}_p^* \times \mathbb{Z}_q^*$.

4.8 Números primos e Teorema dos Números Primos

Se $\pi(x)$ conta os primos menores ou iguais a x , então

$$\pi(x) \sim \frac{x}{\ln x}.$$

Perto de x , a fração de inteiros primos é aproximadamente

$$\frac{1}{\ln x}.$$

Entre candidatos ímpares, a densidade é aproximadamente

$$\frac{2}{\ln x}.$$

Para números de 1024 bits, $\ln(2^{1024}) = 1024 \ln 2 \approx 710$. Portanto, testando apenas ímpares, espera-se encontrar um primo após algumas centenas de candidatos.

4.8.1 Quantos primos têm exatamente 100 algarismos?

O número desejado é

$$\pi(10^{100} - 1) - \pi(10^{99} - 1).$$

Pela aproximação $\pi(x) \approx x / \ln x$,

$$\frac{10^{100}}{100 \ln 10} - \frac{10^{99}}{99 \ln 10} \approx 3.90 \times 10^{97}.$$

Usando os limites fornecidos na lista,

$$\ln x - \frac{3}{2} \leq \frac{x}{\pi(x)} \leq \ln x - \frac{1}{2},$$

e aplicando o limite inferior em 10^{100} e o superior em 10^{99} (e vice-versa), obtém-se aproximadamente

$$3.91 \times 10^{97} \lesssim \#\{\text{primos de 100 algarismos}\} \lesssim 3.93 \times 10^{97}.$$

Trocar $10^d - 1$ por 10^d não afeta os algarismos exibidos nessa escala.

4.9 Miller–Rabin

Para testar um inteiro ímpar $n > 2$, escreva

$$n - 1 = 2^s d, \quad d \text{ ímpar}.$$

Escolha uma base aleatória $a \in \{2, \dots, n - 2\}$ e calcule

$$x = a^d \bmod n.$$

Uma rodada é:

1. se $x = 1$ ou $x = n - 1$, a base não detectou composição;
2. repita no máximo $s - 1$ vezes:

$$x \leftarrow x^2 \bmod n;$$

- se $x = n - 1$, a base não detectou composição;
3. se nenhum valor foi $n - 1$, declare n composto.

Se n é primo, todas as bases válidas passam. Se n é ímpar composto, no máximo $1/4$ das bases são *strong liars*. Com t rodadas independentes:

$$\Pr[\text{composto aceito como primo}] \leq 4^{-t}.$$

O teste é probabilístico no sentido de poder aceitar um composto; quando declara “composto”, a conclusão é certa.

4.9.1 Duas rodadas executadas à mão

Composto $n = 21$, **base** $a = 2$. Escreva

$$20 = 2^2 \cdot 5, \quad s = 2, d = 5.$$

Calcule

$$x = 2^5 \bmod 21 = 11.$$

Não é 1 nem 20. Há apenas mais uma quadratura:

$$x^2 \equiv 11^2 \equiv 16 \pmod{21},$$

que também não é 20. A base 2 testemunha que 21 é composto.

Primo $n = 29$, **base** $a = 2$. Agora

$$28 = 2^2 \cdot 7, \quad 2^7 \equiv 12 \pmod{29}.$$

Na primeira quadratura,

$$12^2 \equiv 144 \equiv 28 \equiv -1 \pmod{29},$$

portanto a rodada passa, como deve ocorrer para toda base válida quando n é primo.

4.9.2 Dois significados de “probabilidade de erro”

Para um **composto fixo** n , uma base aleatória de Miller–Rabin deixa o número passar com probabilidade no máximo $1/4$. Após t bases independentes, o limite é 4^{-t} .

Já a pergunta “qual a chance de o número devolvido pelo gerador ser composto, dado que passou?” é condicional e depende da densidade de primos entre os candidatos. A Tabela 4.3 do HAC fornece limites mais apertados para busca aleatória. Um número de 100 algarismos tem cerca de 333 bits; a tabela não possui exatamente essa linha. Para uma rodada, ela lista 2^{-19} em 300 bits e 2^{-28} em 350 bits, indicando a escala do limite aplicável. Não se deve confundir essa probabilidade posterior com o limite universal de $1/4$ para uma base aplicada a um composto previamente fixado.

4.9.3 Geração de um primo grande

1. gere um candidato uniforme com o bit mais significativo igual a 1;
2. force o bit menos significativo a 1, tornando-o ímpar;
3. elimine divisibilidade por pequenos primos;
4. execute várias rodadas de Miller–Rabin;
5. se falhar, gere outro candidato.

Uma implementação direta de uma rodada segue esta lógica:

```
milller_rabin_rodada(n, a):
    se gcd(a,n) > 1: devolver "composto"
    escrever n-1 = 2^s d, com d ímpar
    x = a^d mod n                # usar square-and-multiply
    se x = 1 ou x = n-1: devolver "passou"
    repetir s-1 vezes:
        x = x^2 mod n
        se x = n-1: devolver "passou"
    devolver "composto"
```

O teste com t rodadas sorteia novas bases e só devolve “provável primo” se todas passarem. Casos $n < 4$, números pares e bases fora do intervalo precisam ser tratados na interface da função.

Para gerar um primo de 1024 bits, o passo 1 deve sortear exatamente nesse intervalo:

$$2^{1023} \leq n < 2^{1024}.$$

Fixar o bit mais significativo garante 1024 bits; fixar o menos significativo elimina candidatos pares. Cada candidato aprovado é um **provável primo**, a menos que depois seja usado um teste que produza prova determinística de primalidade.

4.10 Ordens: exemplos da lista

4.10.1 Grupo aditivo $\mathbb{Z}_{36}^+$

Usando

$$\text{ord}(a) = \frac{36}{\gcd(a, 36)},$$

os elementos ficam agrupados por ordem:

Ordem	Elementos
1	0
2	18
3	12, 24
4	9, 27
6	6, 30
9	4, 8, 16, 20, 28, 32
12	3, 15, 21, 33
18	2, 10, 14, 22, 26, 34
36	1, 5, 7, 11, 13, 17, 19, 23, 25, 29, 31, 35

4.10.2 Grupo multiplicativo $\mathbb{Z}_{36}^*$

$$\mathbb{Z}_{36}^* = \{1, 5, 7, 11, 13, 17, 19, 23, 25, 29, 31, 35\}, \quad \varphi(36) = 12.$$

Ordem	Elementos
1	1

Ordem	Elementos
2	17, 19, 35
3	13, 25
6	5, 7, 11, 23, 29, 31

Nenhum elemento tem ordem 12, portanto $\mathbb{Z}_{36}^*$ não é cíclico.

4.10.3 Grupo $\mathbb{Z}_{37}^*$

Como 37 é primo:

$$|\mathbb{Z}_{37}^*| = 36.$$

Ele é cíclico. Seus geradores são os elementos de ordem 36:

$$2, 5, 13, 15, 17, 18, 19, 20, 22, 24, 32, 35.$$

Há exatamente

$$\varphi(36) = 12$$

geradores, como esperado.

A tabela completa pedida na lista é:

Ordem	Elementos de $\mathbb{Z}_{37}^*$
1	1
2	36
3	10, 26
4	6, 31
6	11, 27
9	7, 9, 12, 16, 33, 34
12	8, 14, 23, 29
18	3, 4, 21, 25, 28, 30
36	2, 5, 13, 15, 17, 18, 19, 20, 22, 24, 32, 35

As quantidades em cada linha são coerentes com a estrutura cíclica: para cada divisor d de 36, há $\varphi(d)$ elementos de ordem d , e a soma dessas quantidades é 36.

4.10.4 Tabela de potências módulo 17

3 tem ordem 16 em $\mathbb{Z}_{17}^*$. A sequência é:

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
$3^i \bmod 17$	1	3	9	10	13	5	15	11	16	14	8	7	4	12	2	6	1

Se $a = 3^j$, então

$$a^{-1} = 3^{16-j} \pmod{17}.$$

Uma tabela de índices transforma inversão em uma consulta e uma subtração de expoentes. Para grupos criptográficos grandes, construir a tabela inteira custaria memória proporcional à ordem do grupo.

5 Parte IV — Criptografia de chave pública

5.1 Visão geral

Em criptografia de chave pública, cada usuário possui:

$$(\text{pk}, \text{sk}) \leftarrow \text{Gen}(1^\lambda).$$

- pk é pública;
- sk permanece secreta;
- qualquer pessoa pode cifrar para o proprietário de pk;
- apenas quem possui sk deve conseguir decifrar.

Isso não elimina a necessidade de autenticar chaves públicas. Sem certificados ou outro mecanismo de autenticação, um adversário pode substituir uma chave e executar ataque man-in-the-middle.

5.2 RSA

5.2.1 Geração de chaves

1. escolha primos grandes e distintos p, q ;
2. calcule

$$n = pq;$$

3. calcule

$$\varphi(n) = (p-1)(q-1);$$

4. escolha e com

$$\gcd(e, \varphi(n)) = 1;$$

5. calcule

$$d \equiv e^{-1} \pmod{\varphi(n)}.$$

Chaves:

$$\text{pk} = (n, e), \quad \text{sk} = (n, d)$$

e, na prática, também se guardam p, q e parâmetros CRT para acelerar a decifração.

5.2.2 Exemplo pequeno de geração, encriptação e decifração

Os números seguintes são didáticos, não seguros. Escolha

$$p = 61, \quad q = 53, \quad n = 3233, \quad \varphi(n) = 60 \cdot 52 = 3120.$$

Tome $e = 17$. Como $\gcd(17, 3120) = 1$, o inverso existe. Euclides estendido fornece

$$d = 17^{-1} \pmod{3120} = 2753,$$

pois $17 \cdot 2753 = 1 + 15 \cdot 3120$. Para $m = 65$:

$$c = 65^{17} \pmod{3233} = 2790,$$

$$\widehat{m} = 2790^{2753} \pmod{3233} = 65.$$

As potências são calculadas com Square-and-Multiply. Esse exemplo reúne a sequência completa: escolher primos, calcular φ , verificar o MDC, obter o inverso, elevar para cifrar e elevar para decifrar.

5.2.3 Encriptação e decifração textbook

Para $m \in \mathbb{Z}_n$:

$$c = m^e \pmod{n}, \quad \widehat{m} = c^d \pmod{n}.$$

5.2.4 Corretude quando $\gcd(m, n) = 1$

Como

$$ed \equiv 1 \pmod{\varphi(n)},$$

existe t tal que

$$ed = 1 + t\varphi(n).$$

Pelo teorema de Euler:

$$m^{\varphi(n)} \equiv 1 \pmod{n}.$$

Logo:

$$c^d \equiv m^{ed} = m^{1+t\varphi(n)} \equiv m(m^{\varphi(n)})^t \equiv m \pmod{n}.$$

5.2.5 Corretude para todo $m \in \mathbb{Z}_n$

A condição $\gcd(m, n) = 1$ é necessária para aplicar Euler diretamente, mas RSA continua correto para mensagens não invertíveis. Prove separadamente módulo p e módulo q .

Módulo p :

- se $m \equiv 0 \pmod{p}$, então $m^{ed} \equiv m \equiv 0 \pmod{p}$;
- caso contrário, Fermat dá $m^{p-1} \equiv 1 \pmod{p}$, e $ed \equiv 1 \pmod{p-1}$ implica $m^{ed} \equiv m \pmod{p}$.

O mesmo vale módulo q . Pelo CRT:

$$m^{ed} \equiv m \pmod{pq}.$$

5.2.6 Decifração acelerada por CRT

Em vez de uma exponenciação módulo n , calcule

$$m_p = c^{d_p} \bmod p, \quad d_p = d \bmod (p-1),$$

$$m_q = c^{d_q} \bmod q, \quad d_q = d \bmod (q-1),$$

e reconstrua o único $m \bmod n$ com esses dois resíduos. As operações módulo p e q usam números com aproximadamente metade dos bits de n e são muito mais rápidas. Implementações também armazenam, por exemplo, $q^{-1} \bmod p$ para fazer a recombinação.

5.2.7 Por que $e = 65537$?

$$65537 = 2^{16} + 1$$

tem representação binária com apenas dois bits 1, tornando a exponenciação pública eficiente por Square-and-Multiply. É grande o bastante para evitar vários problemas associados a expoentes públicos extremamente pequenos e é primo, facilitando obter

$$\gcd(e, \varphi(n)) = 1.$$

$e = 3$ não é automaticamente inseguro quando RSA é implementado com padding correto. O problema é usá-lo com textbook RSA:

- se $m^3 < n$, então $c = m^3$ como inteiro e basta extrair a raiz cúbica;
- enviar a mesma mensagem a três módulos coprimos permite reconstruir m^3 via CRT (ataque de broadcast);
- estrutura algébrica e padding incorreto podem gerar outros ataques.

5.2.8 Fatorar versus inverter RSA

Conhecer p, q permite calcular $\varphi(n)$, obter d e quebrar RSA. Portanto:

$$\text{fatorar } n \implies \text{inverter RSA}.$$

A recíproca não é conhecida em geral: não há prova de que qualquer algoritmo capaz de inverter instâncias RSA necessariamente fatore n . “RSA é tão difícil quanto fatoração” não deve ser afirmado como equivalência demonstrada.

5.2.9 Conhecer $\varphi(n)$ implica fatorar $n = pq$

Como

$$\varphi(n) = (p-1)(q-1) = pq - p - q + 1,$$

temos

$$p + q = n - \varphi(n) + 1.$$

Logo p, q são raízes de

$$x^2 - (n - \varphi(n) + 1)x + n = 0.$$

Para

$$n = 4294049777, \quad \varphi(n) = 4293918720,$$

obtemos

$$p + q = 131058$$

e discriminante

$$\Delta = 131058^2 - 4 \cdot 4294049777 = 256 = 16^2.$$

Portanto:

$$p = \frac{131058 + 16}{2} = 65537, \quad q = \frac{131058 - 16}{2} = 65521.$$

5.2.10 Mensagem não coprime com n

Se uma mensagem aleatória satisfaz

$$\gcd(m, n) \neq 1,$$

então calcular o próprio $\gcd(m, n)$ revela p ou q . Esse é o problema grave; não é uma falha da corretude RSA.

A probabilidade para m uniforme módulo $n = pq$ é:

$$\Pr[\gcd(m, n) \neq 1] = 1 - \frac{\varphi(n)}{n} = \frac{1}{p} + \frac{1}{q} - \frac{1}{pq}.$$

Para primos RSA grandes, é desprezível, mas se ocorrer a fatoração fica imediata.

5.2.11 Procedimento para o exercício com primos de 12 algarismos

1. escolha dois primos distintos p, q com $10^{11} \leq p, q < 10^{12}$;
2. calcule $n = pq$ e confirme que ele tem aproximadamente 24 algarismos;
3. calcule $\varphi(n) = (p-1)(q-1)$;
4. tente $e = 65537$ e confirme $\gcd(e, \varphi(n)) = 1$; se falhar, troque os primos ou escolha outro expoente válido;
5. use Euclides estendido para obter d e verifique $ed \bmod \varphi(n) = 1$;
6. escolha $0 \leq m < n$, calcule $c = m^e \bmod n$ e confirme $c^d \bmod n = m$.

Primos de 12 algarismos são adequados apenas para o exercício. O teste final deve incluir as verificações algébricas; imprimir somente quatro números não mostra que a chave foi gerada corretamente.

5.3 Por que textbook RSA não é seguro?

5.3.1 Determinismo

$$\text{Enc}_{\text{pk}}(m) = m^e \bmod n$$

sempre produz o mesmo cifrado. No jogo CPA, o adversário cifra m_0 publicamente e compara com o desafio.

5.3.2 Maleabilidade multiplicativa

$$\text{Enc}(m_1)\text{Enc}(m_2) \equiv (m_1 m_2)^e \equiv \text{Enc}(m_1 m_2) \pmod{n}.$$

Um adversário altera um cifrado de maneira previsível sem conhecer a mensagem.

5.3.3 Mensagens pequenas e estrutura

Textbook RSA preserva relações algébricas e não introduz aleatoriedade. RSA real precisa de padding/encoding com análise de segurança, como OAEP para encriptação.

5.4 OAEP

OAEP aplica uma transformação Feistel-like, com funções de máscara G e H , antes da operação RSA.

Em forma simplificada, para mensagem codificada M e semente aleatória r :

$$X = M \oplus G(r), \quad Y = r \oplus H(X).$$

O bloco $X\|Y$ é então processado por RSA. Para reverter:

$$r = Y \oplus H(X), \quad M = X \oplus G(r).$$

Objetivos:

- tornar probabilística a encriptação;
- fazer a mensagem inteira depender de todos os bits de X e Y ;
- evitar testes diretos de mensagens;
- impedir o uso simples da estrutura multiplicativa de textbook RSA.

A propriedade *all-or-nothing* significa que perder ou alterar parte do bloco impede recuperar corretamente tanto a semente quanto a mensagem. Em implementações reais, OAEP inclui rótulo, hash do rótulo e regras precisas de codificação; não basta inventar um padding semelhante.

5.5 Diffie–Hellman

Parâmetros públicos:

$$G = \langle g \rangle, \quad |G| = q.$$

Fluxo:

1. Alice escolhe

$$a \xleftarrow{\$} \mathbb{Z}_q$$

e envia $A = g^a$;

2. Bob escolhe

$$b \xleftarrow{\$} \mathbb{Z}_q$$

e envia $B = g^b$;

3. Alice calcula

$$K_A = B^a = g^{ab};$$

4. Bob calcula

$$K_B = A^b = g^{ab}.$$

Uma KDF deve transformar o segredo de grupo em chaves simétricas separadas.

5.5.1 Exemplo completo e verificação

Use o primo $p = 23$ e $g = 5$. Verifica-se que 5 tem ordem 22 módulo 23, logo gera todo o $\mathbb{Z}_{23}^*$. Alice escolhe $a = 6$ e Bob escolhe $b = 15$.

$$A = g^a = 5^6 \bmod 23 = 8, \quad B = g^b = 5^{15} \bmod 23 = 19.$$

Alice recebe B e calcula

$$K_A = B^a = 19^6 \bmod 23 = 2.$$

Bob recebe A e calcula

$$K_B = A^b = 8^{15} \bmod 23 = 2.$$

Os dois chegam ao mesmo valor porque

$$K_A = (g^b)^a = g^{ab} = g^{ba} = (g^a)^b = K_B.$$

Um observador que vê p, g, A, B precisaria calcular g^{ab} sem conhecer a nem b ; essa é exatamente a hipótese CDH. As exponenciações saem por Square-and-Multiply, nunca por multiplicação repetida.

Em uso real, g^{ab} é processado por uma KDF antes de virar chave de sessão. Ele é um elemento de grupo, não uma chave de 128 bits: usar diretamente seus bits desperdiça entropia e pode herdar a estrutura algébrica do grupo.

5.5.2 Por que “estabelecimento”, não “troca”?

Nenhuma das partes escolhe antecipadamente o valor final

$$g^{ab}.$$

Cada uma contribui com um expoente, e a chave surge da combinação. A chave não é simplesmente transportada de uma parte para a outra.

5.5.3 Limitação: falta de autenticação

DH puro resiste a um observador passivo sob hipóteses adequadas, mas não a um adversário ativo. Mallory pode estabelecer uma chave com Alice e outra com Bob. Para impedir MITM, autentique os valores efêmeros com assinaturas, certificados ou uma chave previamente compartilhada.

5.6 Problemas DLP, CDH e DDH

Em $G = \langle g \rangle$:

- **DLP**: dado g e $h = g^x$, encontrar x ;
- **CDH**: dados g^a, g^b , calcular g^{ab} ;
- **DDH**: distinguir

$$(g^a, g^b, g^{ab})$$

de

$$(g^a, g^b, g^c)$$

para c aleatório.

Em geral:

$$\text{resolver DLP} \implies \text{resolver CDH} \implies \text{resolver DDH},$$

mas as recíprocas não são conhecidas em todos os grupos.

5.7 ElGamal

5.7.1 Geração

Em $G = \langle g \rangle$ de ordem q :

$$x \xleftarrow{\$} \mathbb{Z}_q, \quad h = g^x.$$

Chaves:

$$\text{pk} = (G, q, g, h), \quad \text{sk} = x.$$

5.7.2 Encriptação

Para $m \in G$, escolha

$$y \xleftarrow{\$} \mathbb{Z}_q$$

novo e calcule

$$c_1 = g^y, \quad c_2 = m \cdot h^y.$$

O cifrado é

$$c = (c_1, c_2).$$

5.7.3 Decriptação

$$m = \frac{c_2}{c_1^x} = \frac{m \cdot (g^x)^y}{(g^y)^x} = m.$$

ElGamal é a perspectiva não interativa de Diffie–Hellman: $h = g^x$ funciona como a contribuição fixa do receptor, g^y é a contribuição efêmera do remetente, e $h^y = g^{xy}$ é o segredo compartilhado usado como máscara.

Com expoente y novo em cada encriptação e hipótese DDH, ElGamal é CPA-seguro. Reutilizar y relaciona os cifrados:

$$\frac{c_2}{c_2'} = \frac{m}{m'}.$$

5.7.4 Exemplo completo

Com os mesmos parâmetros $p = 23$ e $g = 5$, escolha a chave secreta $x = 6$, logo

$$h = g^x = 5^6 \bmod 23 = 8.$$

Para cifrar $m = 7$, o remetente escolhe o expoente efêmero $y = 3$:

$$c_1 = g^y = 5^3 \bmod 23 = 10,$$

$$c_2 = m \cdot h^y = 7 \cdot 8^3 \bmod 23 = 7 \cdot 6 \bmod 23 = 19.$$

O cifrado é $(c_1, c_2) = (10, 19)$. O receptor calcula

$$c_1^x = 10^6 \bmod 23 = (5^3)^6 = 5^{18} \bmod 23 \equiv 6,$$

e recupera

$$m = c_2 \cdot (c_1^x)^{-1} = 19 \cdot 6^{-1} \bmod 23 = 19 \cdot 4 \bmod 23 = 7,$$

pois $6^{-1} \equiv 4 \pmod{23}$. O valor $y = 3$ precisa ser novo em cada mensagem: repeti-lo produziria

$$\frac{c_2}{c_2'} = \frac{m}{m'},$$

que revela a razão entre os dois textos claros sem qualquer esforço computacional.

5.8 Escolha do grupo e safe primes

Se

$$p = 2q + 1$$

com p, q primos, então p é um *safe prime*. O conjunto dos resíduos quadráticos módulo p forma um subgrupo de ordem q .

Para $b \in \mathbb{Z}_p^*$, defina

$$c = b^2 \bmod p.$$

Então c pertence ao subgrupo de resíduos quadráticos. Como esse subgrupo tem ordem prima q :

- se $c = 1$ (casos $b = \pm 1$), sua ordem é 1;
- caso contrário, sua ordem é q e c é gerador do subgrupo.

Usar o subgrupo de ordem prima facilita a análise e evita elementos de ordem pequena.

5.8.1 b ou c : qual usar?

O elemento b gera um grupo de ordem $2q$ que contém -1 , de ordem 2. Se a implementação aceitar entradas inválidas, um adversário ativo pode forçar o segredo para esse subgrupo pequeno e recuperar informação pelo resíduo do expoente. Já $c = b^2$ pertence ao subgrupo de resíduos quadráticos, de ordem prima q , cujas únicas ordens possíveis são 1 e q .

Por isso c é a escolha mais limpa como parâmetro: elimina subgrupos pequenos e faz a segurança depender de um único parâmetro primo q , que também é o expoente do melhor ataque genérico, $O(\sqrt{q})$.

5.9 Algoritmos para fatoração

5.9.1 Pollard–rho para fatoração

Escolha uma função iterada, por exemplo

$$f(x) = x^2 + c \bmod n.$$

Gere duas seqüências com Floyd:

$$x \leftarrow f(x), \quad y \leftarrow f(f(y)).$$

Em cada passo calcule

$$d = \gcd(|x - y|, n).$$

- $d = 1$: continue;
- $1 < d < n$: d é um fator;
- $d = n$: a tentativa degenerou; troque a semente ou c .

Intuição: módulo de um fator primo p , a sequência entra em ciclo após cerca de $O(\sqrt{p})$ passos pelo fenômeno do aniversário. Uma colisão módulo p faz $p \mid (x - y)$, revelando-o pelo gcd.

5.9.2 Exemplo executado

Fatore

$$n = 8051.$$

Use $f(x) = x^2 + 1 \bmod n$, semente $x_0 = 2$ e o par de Floyd ($x \leftarrow f(x)$, $y \leftarrow f(f(y))$):

passo	x	y	$\gcd(x - y , n)$
1	5	26	1
2	26	7474	1
3	677	871	97

No terceiro passo 97 divide n , então o algoritmo para e devolve

$$8051 = 97 \cdot 83.$$

A conta é curta porque n é pequeno; o que importa é a taxa de crescimento. O número esperado de passos é $O(\sqrt{p})$, onde p é o menor fator primo, e não $O(\sqrt{n})$: a colisão que o algoritmo procura ocorre módulo p , e é o aniversário aplicado ao grupo gerado por p que determina o custo.

5.10 Algoritmos para log discreto

5.10.1 Busca exaustiva

Teste

$$g^0, g^1, \dots$$

até encontrar h . Custo $O(q)$ para grupo de ordem q .

5.10.2 Baby-Step/Giant-Step

Usa decomposição do expoente e uma tabela de aproximadamente $\sqrt{q}$ elementos:

$$\text{tempo } O(\sqrt{q}), \quad \text{memória } O(\sqrt{q}).$$

5.10.3 Pollard–rho para DLP

Mantenha estados com representação conhecida

$$X_i = g^{a_i} h^{b_i}.$$

Uma função de iteração atualiza X_i, a_i, b_i . Quando se encontra colisão

$$X_i = X_j,$$

temos, escrevendo $h = g^x$:

$$g^{a_i + b_i x} = g^{a_j + b_j x}.$$

Logo:

$$(b_i - b_j)x \equiv a_j - a_i \pmod{q}.$$

Se $b_i - b_j$ é invertível módulo q :

$$x \equiv (a_j - a_i)(b_i - b_j)^{-1} \pmod{q}.$$

Complexidade esperada:

$$O(\sqrt{q})$$

operações de grupo e memória constante. Uma colisão degenerada exige reiniciar.

5.10.4 Index Calculus

Index Calculus explora a representação específica dos elementos de $\mathbb{Z}_p^*$ como inteiros que podem fatorar em uma base de fatores pequenos.

1. escolha uma base

$$\mathcal{B} = \{p_1, \dots, p_t\};$$

2. procure expoentes r para os quais

$$g^r \bmod p = \prod_i p_i^{e_i}$$

seja *smooth*;

3. obtenha equações lineares

$$r \equiv \sum_i e_i \log_g(p_i) \pmod{p-1};$$

4. resolva os logs dos elementos da base;

5. combine h com uma potência de g até obter outro valor smooth e recupere $\log_g h$.

É subexponencial em $\mathbb{Z}_p^*$ e supera ataques genéricos. Não funciona diretamente em grupos de curvas elípticas porque pontos da curva não possuem uma fatoração smooth análoga. Por isso, para parâmetros bem escolhidos de ECC, os melhores ataques genéricos continuam da ordem de $\sqrt{q}$.

5.11 Curvas elípticas

Sobre $\mathbb{F}_p$, uma curva curta de Weierstrass é:

$$E: y^2 = x^3 + ax + b \pmod{p},$$

com condição de não singularidade

$$4a^3 + 27b^2 \not\equiv 0 \pmod{p}.$$

Os pontos da curva, mais o ponto no infinito $\mathcal{O}$, formam um grupo abeliano sob adição geométrica/algebraica.

5.11.1 Soma de pontos

Para $P = (x_1, y_1)$ e $Q = (x_2, y_2)$, $P \neq \pm Q$:

$$\lambda = \frac{y_2 - y_1}{x_2 - x_1} \pmod{p}.$$

Para duplicação $P = Q$:

$$\lambda = \frac{3x_1^2 + a}{2y_1} \pmod{p}.$$

Então:

$$x_3 = \lambda^2 - x_1 - x_2 \pmod{p},$$

$$y_3 = \lambda(x_1 - x_3) - y_1 \pmod{p}.$$

As divisões significam multiplicação pelo inverso modular.

5.11.2 Exemplo executado

Considere

$$E : y^2 = x^3 + 2x + 2 \pmod{17}.$$

Como

$$4a^3 + 27b^2 = 32 + 108 = 140 \equiv 4 \pmod{17} \neq 0,$$

a curva é não singular. Ela tem 18 pontos afins mais o ponto no infinito, totalizando

$$|E| = 19,$$

um número primo. Tome

$$P = (5, 1),$$

que está na curva, pois $5^3 + 2 \cdot 5 + 2 = 137 \equiv 1 \pmod{17}$. Aqui $P = Q$, então usamos a fórmula de duplicação:

$$\lambda = \frac{3 \cdot 5^2 + 2}{2 \cdot 1} = \frac{77}{2} \equiv 9 \cdot 2^{-1} \equiv 9 \cdot 9 \equiv 13 \pmod{17},$$

pois $2^{-1} \equiv 9 \pmod{17}$. Segue

$$x_3 = \lambda^2 - 2x_1 = 169 - 10 = 159 \equiv 6 \pmod{17},$$

$$y_3 = \lambda(x_1 - x_3) - y_1 = 13(5 - 6) - 1 = -14 \equiv 3 \pmod{17}.$$

Portanto

$$2P = (6, 3).$$

Continuando, $3P = (10, 6)$. Como $|E| = 19$ é primo, todo ponto não nulo tem ordem 19; em particular os múltiplos

$$P, 2P, 3P, \dots, 18P$$

são todos distintos e

$$19P = \mathcal{O}.$$

É essa ordem grande e prima que torna o grupo útil: o ECDLP com base P custa cerca de $\sqrt{19}$ passos neste exemplo e $\sqrt{q}$ no caso real.

5.11.3 ECDLP

Dado um ponto base P de ordem grande e

$$Q = xP,$$

o ECDLP pede encontrar o escalar x . Falar “log discreto” é estranho apenas porque a notação do grupo é aditiva: não aparece uma potência g^x , mas a multiplicação escalar xP é a operação análoga.

5.12 Fatores de trabalho e tamanhos de chave

Problema/grupo	Melhor família clássica relevante	Consequência
fatoração/RSA	algoritmos subexponenciais	módulos precisam ser grandes
DLP em $\mathbb{Z}_p^*$	Index Calculus subexponencial	p precisa ser grande
ECDLP bem parametrizado	Pollard-rho $O(\sqrt{q})$	chaves ECC são menores

Comparação clássica aproximada de força:

Força	RSA	DH/DSA em campo finito	ECC	Simétrica
112 bits	2048	$L = 2048, N = 224$	224–255	pelo menos 112
128 bits	3072	$L = 3072, N = 256$	256–383	AES-128
192 bits	7680	$L = 7680, N = 384$	384–511	AES-192
256 bits	15360	$L = 15360, N = 512$	512+	AES-256

L é o tamanho do módulo e N a ordem do subgrupo. A tabela compara segurança **clássica**; um computador quântico de grande escala mudaria a comparação para RSA, DH e ECC.

6 Parte V — Hashes, MACs e assinaturas

6.1 Objetivos diferentes

Primitivo	Usa segredo?	Quem verifica?	Objetivo
CRC	não	qualquer pessoa	detectar erros acidentais
hash criptográfico	não	qualquer pessoa	resumo com propriedades contra ataques
MAC	sim, chave compartilhada	quem conhece a chave	integridade e autenticação simétricas
assinatura digital	chave privada para assinar	qualquer pessoa com a chave pública	integridade, origem e verificabilidade pública

O **CRC** (*cyclic redundancy check*) é o nível mais fraco da tabela. A mensagem é vista como um polinômio $M(x)$ sobre $GF(2)$, e o valor transmitido é o resto

$$R(x) = M(x) \bmod G(x)$$

para um polinômio gerador $G(x)$ fixo e público. É um código de detecção de erro, não um mecanismo de segurança. Em particular ele é **linear**: alterar M de forma controlada altera R de forma previsível, então um adversário recalcula a verificação sem conhecer nada secreto.

Um hash não autentica sozinho: qualquer pessoa que altera m também pode calcular $H(m')$. Autenticação exige uma chave (MAC) ou assinatura.

6.2 Função hash

Uma função hash mapeia entradas arbitrárias para saída de tamanho fixo:

$$H : \{0, 1\}^* \longrightarrow \{0, 1\}^n.$$

6.2.1 Preimage resistance / one-wayness

Dado y , deve ser difícil encontrar x tal que

$$H(x) = y.$$

Ataque genérico: aproximadamente 2^n avaliações.

6.2.2 Second-preimage resistance / weak collision resistance

Dado x , deve ser difícil encontrar $x' \neq x$ tal que

$$H(x') = H(x).$$

Ataque genérico: aproximadamente 2^n avaliações.

6.2.3 Collision resistance / strong collision resistance

Deve ser difícil encontrar **qualquer** par $x \neq x'$ tal que

$$H(x) = H(x').$$

Ataque genérico: aproximadamente $2^{n/2}$ avaliações, pelo paradoxo do aniversário.

Collision resistance implica, em contextos usuais, uma garantia mais forte que permitir ao adversário receber uma primeira mensagem fixa. Não confunda: em colisão, o atacante escolhe as duas mensagens; em second preimage, uma delas já foi fixada.

6.3 Paradoxo do aniversário

Escolha q valores uniformes e independentes em um conjunto de tamanho N . A probabilidade exata de nenhuma colisão é:

$$\Pr[\text{sem colisão}] = \prod_{i=0}^{q-1} \left(1 - \frac{i}{N}\right).$$

Logo:

$$\Pr[\text{colisão}] = 1 - \prod_{i=0}^{q-1} \left(1 - \frac{i}{N}\right).$$

Para $q \ll N$:

$$\Pr[\text{colisão}] \approx 1 - \exp\left(-\frac{q(q-1)}{2N}\right).$$

Para probabilidade 1/2:

$$q \approx \sqrt{2N \ln 2} \approx 1.177\sqrt{N}.$$

6.3.1 Exemplo com 36500 possibilidades

$$\sqrt{2 \cdot 36500 \ln 2} \approx 224.94.$$

Pelo produto exato:

$$\Pr[\text{colisão com 225}] \approx 0.49934,$$

$$\Pr[\text{colisão com 226}] \approx 0.50243.$$

Portanto, o menor q com probabilidade pelo menos 1/2 é

$$q = 226.$$

6.3.2 Exemplo das carteiras

Com $P = 10^{10}$ pessoas, $A = 10^7$ endereços por pessoa e espaço $N = 2^{160}$, o número de pares de endereços pertencentes a pessoas diferentes é aproximadamente

$$\binom{P}{2} A^2.$$

Como a probabilidade é muito pequena:

$$\Pr[\text{alguma coincidência entre pessoas}] \approx \frac{\binom{P}{2} A^2}{2^{160}} \approx 3.42 \times 10^{-15}.$$

O número total de endereços é enorme, mas 2^{160} é muito maior.

O mesmo fenômeno governa os ataques de log discreto e fatoração. Pollard–rho procura uma colisão dentro do grupo gerado pelo fator p (fatoração) ou dentro do grupo onde se procura o log discreto (DLP). Por isso seus custos são $O(\sqrt{p})$ e $O(\sqrt{q})$: a escala do aniversário aplicada ao tamanho do subgrupo **efetivo**, e não ao tamanho de n nem ao de p por inteiro.

6.4 Ataque de colisão contra hash

Para hash de n bits:

1. gere mensagens distintas;
2. calcule seus hashes;

3. armazene e procure duas saídas iguais;
4. após cerca de $2^{n/2}$ tentativas, uma colisão torna-se provável.

Consequência: um hash de n bits fornece, no máximo, cerca de $n/2$ bits de segurança contra colisões. Para obter aproximadamente 128 bits contra colisão, deseja-se saída de pelo menos 256 bits.

Essa observação separou definitivamente três noções na definição de segurança de hashes. “Difícil inverter” não implica automaticamente “difícil encontrar colisão”.

6.5 Hashes conhecidos

- **MD5:** colisões práticas; não usar para segurança;
- **SHA-1:** resistência a colisão quebrada e em retirada;
- **SHA-2:** inclui SHA-256 e SHA-512; família recomendada;
- **SHA-3:** construção baseada em Keccak; alternativa padronizada à família SHA-2.

A classificação depende da propriedade e da aplicação. Um algoritmo quebrado para colisões não deve ser usado em assinaturas, mesmo que ataques de preimage ainda sejam muito mais caros.

6.6 MAC

Um MAC é uma tripla:

$$(\text{Gen}, \text{Mac}, \text{Vrfy}).$$

$$k \leftarrow \text{Gen}(1^\lambda), \quad t \leftarrow \text{Mac}_k(m),$$

$$\text{Vrfy}_k(m, t) \in \{0, 1\}.$$

Corretude:

$$\Pr[\text{Vrfy}_k(m, \text{Mac}_k(m)) = 1] = 1.$$

Segurança usual: mesmo após consultar tags de mensagens escolhidas, um adversário não deve produzir uma tag válida para uma mensagem nova. Essa é uma forma de unforgeability under chosen-message attack.

6.6.1 Construções perigosas

Não se deve assumir que

$$H(k \| m)$$

é um MAC seguro para qualquer hash. Em construções Merkle–Damgård, pode haver ataque de extensão de comprimento. Outras concatenações ingênuas também carecem de prova e podem ter ambiguidades.

6.6.2 HMAC

HMAC usa duas aplicações do hash com separação de domínio:

$$\text{HMAC}_K(m) = H((K' \oplus \text{opad}) \| H((K' \oplus \text{ipad}) \| m)).$$

K' é a chave ajustada ao tamanho do bloco do hash. Os valores ipad e opad separam o domínio interno do externo.

6.6.3 CBC-MAC

CBC-MAC básico processa blocos como CBC com IV fixo e usa o último bloco como tag. A construção básica é segura apenas para mensagens de tamanho fixo. Usá-la diretamente em tamanhos variáveis permite forjar por extensão. Variantes padronizadas, como CMAC, corrigem esse problema.

6.7 Assinaturas digitais

Um esquema de assinatura é:

$$(\text{Gen}, \text{Sign}, \text{Vrfy}).$$

$$(\text{pk}, \text{sk}) \leftarrow \text{Gen}(1^\lambda),$$

$$\sigma \leftarrow \text{Sign}_{\text{sk}}(m), \quad \text{Vrfy}_{\text{pk}}(m, \sigma) \in \{0, 1\}.$$

Corretude:

$$\Pr[\text{Vrfy}_{\text{pk}}(m, \text{Sign}_{\text{sk}}(m)) = 1] = 1.$$

Segurança típica, EUF-CMA: após obter assinaturas de mensagens escolhidas, o adversário não consegue produzir uma assinatura válida para uma mensagem nova.

6.7.1 MAC versus assinatura

MAC	Assinatura
chave compartilhada	par pública/privada
qualquer verificador também pode forjar	verificador público não deve assinar
não prova autoria a terceiros	permite verificabilidade pública
normalmente mais rápido	normalmente mais caro

“Não repúdio” depende também de proteção da chave, identidade/certificado, política e contexto jurídico; não é produzido apenas pela equação criptográfica.

6.8 Tipos de ataque contra assinaturas

6.8.1 Pelo acesso do atacante

1. **key-only**: conhece apenas a chave pública;
2. **known-message**: observa mensagens e assinaturas;
3. **chosen-message**: escolhe mensagens a serem assinadas;
4. **adaptive chosen-message**: escolhe cada consulta com base nas respostas anteriores.

6.8.2 Pelo objetivo alcançado

1. **existential forgery**: produz uma assinatura válida para alguma mensagem nova;
2. **selective forgery**: forja uma mensagem-alvo escolhida;
3. **universal forgery**: consegue assinar qualquer mensagem;
4. **total break**: recupera a chave secreta.

EUF-CMA já considera vitória uma única falsificação existencial após consultas adaptativas.

6.9 Assinatura RSA textbook

Assinatura:

$$\sigma = m^d \bmod n.$$

Verificação:

$$\sigma^e \stackrel{?}{=} m \pmod{n}.$$

6.9.1 Exemplo com números pequenos

Reutilize a chave do exemplo RSA de encriptação, $n = 3233$, $e = 17$, $d = 2753$. Para assinar $m = 65$,

$$\sigma = m^d \bmod n = 65^{2753} \bmod 3233 = 588.$$

A verificação devolve

$$\sigma^e \bmod n = 588^{17} \bmod 3233 = 65 = m.$$

Compare com a encriptação OAEP: a mesma relação c^d e m^e sustenta os dois usos, mas com papéis invertidos. Como assinatura, não é preciso aleatoriedade para quem verifica; é preciso encoding seguro para quem assina, caso contrário valem as falsificações abaixo.

6.9.2 Fraquezas

Falsificação existencial: escolha σ arbitrário e defina

$$m = \sigma^e \bmod n.$$

Então (m, σ) verifica.

Maleabilidade multiplicativa: de assinaturas

$$\sigma_1 = m_1^d, \quad \sigma_2 = m_2^d,$$

segue:

$$\sigma_1 \sigma_2 \equiv (m_1 m_2)^d \pmod{n}.$$

Na prática usa-se hash-and-sign com encoding seguro e separação de domínio, por exemplo RSA-PSS. Apenas substituir m por $H(m)$ sem analisar o encoding e o modelo não justifica automaticamente segurança.

6.10 Assinatura ElGamal

Parâmetros:

$$p \text{ primo}, \quad g \text{ gerador de } \mathbb{Z}_p^*.$$

Chave secreta e pública:

$$x \xleftarrow{\$} \{1, \dots, p-2\}, \quad y = g^x \pmod{p}.$$

Para assinar o resumo $h = H(m)$:

1. escolha nonce

$$k \in \mathbb{Z}_{p-1}^*$$

2. calcule de forma que $\gcd(k, p-1) = 1$;

$$r = g^k \pmod{p};$$

3. calcule

$$s \equiv k^{-1}(h - xr) \pmod{p-1}.$$

A assinatura é (r, s) . Verificação:

$$g^h \stackrel{?}{\equiv} y^r r^s \pmod{p}.$$

Corretude:

$$y^r r^s \equiv g^{xr} g^{ks} = g^{xr+ks} \equiv g^h \pmod{p},$$

pois

$$ks \equiv h - xr \pmod{p-1}.$$

O nonce k precisa ser secreto, imprevisível e nunca reutilizado.

6.10.1 Exemplo com números pequenos

Use $p = 23$, $g = 2$ e a chave secreta $x = 6$, logo

$$y = g^x = 2^6 \pmod{23} = 18.$$

Para o resumo $H(m) = 7$, escolha o nonce $k = 5$; como $\gcd(5, 22) = 1$, ele é válido. Então

$$r = g^k \pmod{p} = 2^5 \pmod{23} = 9.$$

Como $k^{-1} \equiv 9 \pmod{22}$,

$$s \equiv k^{-1}(H(m) - xr) \equiv 9(7 - 6 \cdot 9) \equiv 9 \cdot 19 \equiv 17 \pmod{22}.$$

A assinatura é $(r, s) = (9, 17)$. A verificação calcula

$$y^r r^s \pmod{p} = 18^9 \cdot 9^{17} \pmod{23} = 13,$$

$$g^{H(m)} \pmod{p} = 2^7 \pmod{23} = 13,$$

e aceita, pois os dois lados coincidem. Note que aqui s foi calculado módulo $p - 1 = 22$, e não módulo p ; confundir os dois módulos é o erro mais comum ao reproduzir o esquema.

6.11 DSA

Parâmetros:

- p primo grande;
- q primo com $q \mid (p - 1)$;
- g de ordem q em $\mathbb{Z}_p^*$.

Chaves:

$$x \xleftarrow{\$} \mathbb{Z}_q^*, \quad y = g^x \bmod p.$$

6.11.1 Assinatura

Escolha nonce

$$k \xleftarrow{\$} \mathbb{Z}_q^*.$$

Calcule:

$$r = (g^k \bmod p) \bmod q,$$

$$s = k^{-1}(H(m) + xr) \bmod q.$$

Se $r = 0$ ou $s = 0$, escolha outro k .

6.11.2 Verificação

$$w = s^{-1} \bmod q,$$

$$u_1 = H(m)w \bmod q, \quad u_2 = rw \bmod q,$$

$$v = (g^{u_1} y^{u_2} \bmod p) \bmod q.$$

Aceite se

$$v = r.$$

6.11.3 Exemplo com números pequenos

Escolha $q = 11$ e $p = 23$, pois $11 \mid 22$. Tome $g = 2$, de ordem 11 em $\mathbb{Z}_{23}^*$, e $x = 3$, logo

$$y = g^x = 2^3 \bmod 23 = 8.$$

Para $H(m) = 7$, escolha $k = 5$. Então

$$r = (g^k \bmod p) \bmod q = (2^5 \bmod 23) \bmod 11 = 9 \bmod 11 = 9,$$

$$s = k^{-1}(H(m) + xr) \bmod q = 5^{-1}(7 + 3 \cdot 9) \bmod 11 = 9 \cdot 34 \bmod 11 = 9.$$

A verificação calcula

$$w = s^{-1} \bmod q = 9^{-1} \bmod 11 = 5,$$

$$u_1 = H(m)w \bmod q = 7 \cdot 5 \bmod 11 = 2, \quad u_2 = rw \bmod q = 9 \cdot 5 \bmod 11 = 1,$$

$$v = (g^{u_1} y^{u_2} \bmod p) \bmod q = (2^2 \cdot 8 \bmod 23) \bmod 11 = 9 \bmod 11 = 9.$$

Como $v = r = 9$, a assinatura é aceita. Todo o cálculo de s, u_1, u_2, v ocorre módulo $q = 11$, e só a exponenciação $g^{u_1} y^{u_2}$ ocorre módulo p ; é exatamente essa separação que a otimização explora.

6.11.4 Por que a otimização é segura?

DSA trabalha no subgrupo de ordem prima q , embora os elementos sejam representados módulo p . Expoentes e componentes da assinatura têm tamanho relacionado a q , reduzindo custo e tamanho. A segurança do ataque genérico ao DLP é governada pela ordem q :

$$O(\sqrt{q}).$$

Escolher q com tamanho adequado mantém a segurança mesmo quando $q \ll p$.

6.12 Reutilização do nonce em DSA

Se o mesmo k assina m_1, m_2 , então r se repete:

$$s_1 = k^{-1}(H(m_1) + xr) \pmod{q},$$

$$s_2 = k^{-1}(H(m_2) + xr) \pmod{q}.$$

Subtraindo:

$$s_1 - s_2 \equiv k^{-1}(H(m_1) - H(m_2)) \pmod{q}.$$

Logo:

$$k \equiv (H(m_1) - H(m_2))(s_1 - s_2)^{-1} \pmod{q},$$

e depois:

$$x \equiv (s_1 k - H(m_1))r^{-1} \pmod{q}.$$

Um único nonce repetido compromete toda a chave privada.

6.13 ECDSA e padrões de curvas

ECDSA transporta a estrutura de DSA para o grupo de pontos de uma curva elíptica. A multiplicação

$$g^k$$

é substituída pela multiplicação escalar

$$kG.$$

Famílias conhecidas incluem curvas NIST P, secp256k1, Curve25519 e Ed25519. Houve preocupação histórica com constantes pouco transparentes em alguns processos de padronização. Isso não equivale a uma quebra pública das curvas NIST P. O caso de suspeita de backdoor mais conhecido foi Dual_EC_DRBG, um gerador pseudoaleatório baseado em curvas, posteriormente retirado; ele não é o mesmo que ECDSA.

Curve25519/Ed25519 são alternativas populares com desenho e parâmetros publicamente explicados, além de fórmulas que facilitam implementações resistentes a várias classes de erro.

7 Parte VI — Revisão orientada à prova

7.1 O que precisa de definição, mecanismo e ataque

Assunto	Definição obrigatória	Mecanismo/cálculo	Ataque ou limitação
PRG	indistinguibilidade	pseudo-OTP	enumeração não é PPT
CPA	jogo e vantagem	oráculo de encriptação	cifra determinística
Feistel	rodada	ida e inversão	poucas rodadas
AES	cifra de bloco/SPN	quatro transformações	não confundir bloco e chave
CBC	encadeamento	fórmulas Enc/Dec	IV previsível/maleabilidade
CTR	keystream por contador	XOR	nonce repetido
GCM	AEAD	CTR + GHASH	nonce repetido
Miller–Rabin	testemunha forte	sequência de quadraturas	falso “provavelmente primo”

Assunto	Definição obrigatória	Mecanismo/cálculo	Ataque ou limitação
RSA	trapdoor permutation textbook	Gen/Enc/Dec	determinismo e maleabilidade
OAEP	encoding probabilístico	duas máscaras	implementação incorreta
DH	estabelecimento	g^a, g^b, g^{ab}	MITM
ElGamal	PKE em grupo	(g^y, mh^y)	nonce repetido/maleabilidade
Pollard- ρ	ataque genérico	colisão	$O(\sqrt{q})$
Index Calculus	base de fatores	relações <i>smooth</i>	não se transfere a ECC
hash	três resistências	aniversário	colisão em $2^{n/2}$
CRC	polinômio gerador	divisão em $GF(2)$	linear, não é criptográfico
MAC	autenticação simétrica	HMAC/CBC-MAC	construção ingênua
assinatura	EU-F-CMA	Sign/Vrfy	nonce/textbook
DSA	assinatura em sub-grupo primo	r, s reduzidos módulo q	reutilização de nonce

7.2 Confusões que precisam estar resolvidas

1. **Sigilo perfeito versus computacional:** igualdade exata para adversário ilimitado versus vantagem negligenciável para PPT.
2. **PRG versus aleatoriedade real:** os suportes são muito diferentes; apenas o teste eficiente é impedido.
3. **PRF versus PRP:** função aleatória pode colidir; permutação não.
4. **Stream cipher versus CTR:** CTR transforma uma cifra de bloco em keystream.
5. **IV versus nonce:** requisitos dependem do modo; CBC clássico pede IV imprevisível, CTR/GCM exigem unicidade.
6. **Hash versus MAC:** hash não possui segredo nem autentica origem.
7. **Colisão versus second preimage:** na colisão o adversário escolhe as duas entradas.
8. **Fatoração versus RSA:** fatorar quebra RSA; equivalência geral não é provada.
9. **DLP versus DDH:** decidir se uma tupla é Diffie-Hellman pode ser mais fácil que recuperar o expoente em alguns grupos.
10. **Correção RSA e $\gcd(m, n)$:** RSA continua correto; uma mensagem não coprima revela fator pelo gcd.
11. **Encriptação versus assinatura RSA:** os expoentes aparecem em ordem oposta, mas segurança real exige encodings diferentes.
12. **Autenticidade versus não repúdio:** não repúdio envolve contexto além do algoritmo.
13. **CRC versus hash:** CRC detecta erro accidental e é linear; hash criptográfico é desenhado contra um adversário.
14. **Módulo p versus $p-1$ na assinatura ElGamal:** r é calculado módulo p , mas s é calculado módulo $p-1$; em DSA, é r e s que são reduzidos módulo q .

7.3 Fórmulas que devem sair sem consulta

7.3.1 CBC

$$c_0 = IV, \quad c_i = E_k(m_i \oplus c_{i-1}), \quad m_i = D_k(c_i) \oplus c_{i-1}.$$

7.3.2 CTR

$$z_i = E_k(N \parallel \langle i \rangle), \quad c_i = m_i \oplus z_i.$$

7.3.3 Euler

$$\gcd(a, n) = 1 \implies a^{\varphi(n)} \equiv 1 \pmod{n}.$$

7.3.4 RSA

$$n = pq, \quad d = e^{-1} \bmod \varphi(n), \quad c = m^e \bmod n, \quad m = c^d \bmod n.$$

7.3.5 Diffie-Hellman

$$A = g^a, \quad B = g^b, \quad K = B^a = A^b = g^{ab}.$$

7.3.6 ElGamal

$$h = g^x, \quad (c_1, c_2) = (g^y, mh^y), \quad m = c_2/c_1^x.$$

7.3.7 Aniversário

$$\Pr[\text{colisão}] = 1 - \prod_{i=0}^{q-1} \left(1 - \frac{i}{N}\right) \approx 1 - e^{-q(q-1)/(2N)}.$$

7.3.8 Assinatura ElGamal

$$r = g^k \bmod p, \quad s = k^{-1}(H(m) - xr) \bmod (p-1),$$

$$g^{H(m)} \stackrel{?}{\equiv} y^r r^s \pmod{p}.$$

7.3.9 DSA

$$r = (g^k \bmod p) \bmod q, \quad s = k^{-1}(H(m) + xr) \bmod q.$$

7.4 Perguntas de autoavaliação

7.4.1 Fundamentos

1. Por que o Lema 2.5 e a Definição 2.6 expressam o mesmo objetivo?
2. Por que $1/\lambda^{100}$ não é negligenciável?
3. Como provar que a soma de duas funções negligenciáveis é negligenciável?
4. Como duas distribuições podem ser estatisticamente distantes e computacionalmente indistinguíveis?
5. Quais são os cinco elementos de uma prova por redução?
6. Por que encriptação determinística não pode ser CPA-segura?

7.4.2 Simétrica

7. Por que a função de rodada de Feistel não precisa ser invertível?
8. Quais componentes do AES garantem a inversão?
9. Por que DES está obsoleto mesmo sem ataque estrutural prático melhor que força bruta?
10. Qual requisito muda de CBC para CTR em relação ao IV/nonce?
11. Por que Chained CBC pode falhar mesmo parecendo uma única mensagem CBC?
12. O que GCM autentica além do texto cifrado?

7.4.3 Matemática

13. Como Euclides estendido produz um inverso modular?
14. Quando “dividir” uma congruência é permitido?
15. Qual a diferença entre ordem do grupo e ordem de elemento?
16. Como verificar se um elemento é gerador quando a fatoração de $|G|$ é conhecida?
17. Por que Fermat não basta como teste de primalidade?
18. Qual é o erro máximo após t rodadas independentes de Miller–Rabin?

7.4.4 Chave pública

19. Por que conhecer $\varphi(n)$ permite fatorar um módulo RSA $n = pq$?
20. Por que $e = 3$ é perigoso em textbook RSA?
21. O que se sabe e o que não se sabe sobre fatoração versus inversão RSA?
22. Como OAEP introduz randomização e all-or-nothing?
23. Por que DH puro não autentica as partes?
24. Como ElGamal deriva diretamente de DH?
25. Compare Pollard–rho e Index Calculus em tempo, memória e grupos aplicáveis.
26. Por que ECC usa chaves menores para a mesma força clássica?

7.4.5 Hashes e autenticação

27. Diferencie preimage, second-preimage e collision resistance.
28. Derive a aproximação do paradoxo do aniversário.
29. Por que saída de 256 bits oferece cerca de 128 bits contra colisão?
30. Por que $H(k\|m)$ não deve ser adotado como MAC sem análise?
31. Diferencie MAC e assinatura quanto às pessoas capazes de verificar e forjar.
32. Mostre duas falsificações possíveis em assinatura RSA textbook.
33. Demonstre a equação de verificação da assinatura ElGamal.
34. Derive a recuperação de k e x quando DSA reutiliza nonce.

7.5 Referências por assunto

Assunto	Katz–Lindell, 3ª ed.
sigilo perfeito	Cap. 2
segurança computacional, PRG, CPA e modos	§§3.1–3.6
MACs	Cap. 4
GCM/autenticação	Cap. 5
hashes e aniversário	Cap. 6 e Apêndice A.4
Feistel, DES e AES	§§7.2.2–7.2.5
grupos, primos, RSA e ECC	Cap. 9
Pollard, DLP, Index Calculus e tamanhos	Cap. 10
Diffie–Hellman	§11.3
ElGamal e RSA/OAEP	§§12.4–12.5
assinaturas RSA, DSA e ECDSA	§§13.4–13.5
Euclides e exponenciação modular	Apêndice B

Referências oficiais complementares:

- NIST FIPS 197, *Advanced Encryption Standard (AES)*: <https://doi.org/10.6028/NIST.FIPS.197-upd1>

- NIST SP 800-57 Part 1 Rev. 5, *Recommendation for Key Management*: <https://doi.org/10.6028/NIST.SP.800-57pt1r5>
- NIST, política sobre funções hash: <https://csrc.nist.gov/projects/hash-functions/nist-policy-on-hash-functions>